

Data, EDA and Supervised Learning

Day 2: Detailed learning notes

From inspecting a table to explaining a model's predictions and measuring its mistakes.

Three notebooks, one connected lesson

NumPy and Pandas; exploring and classifying Titanic passengers; and predicting California district house values.

How to use these notes

Open the matching notebook while you read. Run each example, inspect its output, and work through the questions. Code explanations cover what an operation does, why it is used, and what its result means. Prepared from the supplied 33-slide Day 2 deck, all three notebooks, and the speaker

notes. The examples and model results were checked by executing the supplied code.

Contents

1	Starting Day 2	5
1.1	The question for today	5
1.2	The three notebooks and reading order	5
1.3	Before running the examples	5
1.4	How to read the checked results	6
2	NumPy: calculations on arrays	6
2.1	Importing the library	6
2.2	Doubling prices and understanding vectorisation	6
2.3	The speed experiment	7
2.4	Creating several kinds of arrays	8
2.5	Shape, rows, columns, and indexing	8
2.6	Arithmetic and broadcasting	9
2.7	Mean, maximum, and standard deviation	9
2.8	Boolean filtering	9
2.9	The exam-score example and the meaning of axis	10
2.10	Notebook 1, Exercise 1: complete solution	11
3	Pandas: reading and working with tables	11
3.1	DataFrames, Series, and the index	11
3.2	Loading the Titanic table	12
3.3	The first inspection commands	12
3.4	Selecting one column or several columns	13
3.5	Filtering passenger rows	13
3.6	Adding family size	14
3.7	Group and summarise	14
3.8	Counting categories and sorting rows	14
3.9	Notebook 1, Exercise 2: full solution	15
4	Exploratory data analysis with the Titanic	15
4.1	The six questions behind EDA	15
4.2	Read the passenger fields before plotting	16
4.3	Load a fresh copy and inspect it	16
4.4	A count plot: how many examples are in each category?	17
4.5	Using hue to split the counts	17
4.6	A bar plot of a mean: survival by ticket class	18

4.7	A histogram: the distribution of ages	18
4.8	A box plot: fare spread and unusual values	19
4.9	A correlation heatmap	19
4.10	Choosing the plot for the question	21
4.11	Answering the EDA discussion questions	21
5	Cleaning, encoding, features, and splitting	22
5.1	First remove information that gives away the answer	22
5.2	Missing values: what are the options?	22
5.3	The imputation code in the supplied notebook	23
5.4	Family size and travelling alone	23
5.5	Binary encoding and category codes	24
5.6	Why scaling changes the numerical problem	24
5.7	Separating the target and splitting paired rows	25
5.8	Fit the scaler once, then reuse it	25
5.9	A preparation order to use in your own projects	26
6	The housing dataset: a regression problem	27
6.1	Understand what one row represents	27
6.2	Load and inspect the source table	27
6.3	Inspecting the target distribution and its cap	28
6.4	A geographical plot from coordinates	28
6.5	Looking for numerical relationships	29
6.6	Filling the missing bedroom totals	30
6.7	Why ratios can describe a district differently	30
6.8	Encoding ocean categories	31
6.9	The housing split and scaler	31
7	Regression metrics: measuring numerical mistakes	32
7.1	Residuals and why the signs matter	32
7.2	Mean absolute error	32
7.3	Root mean squared error	32
7.4	Understanding R-squared	33
7.5	The reporting helper, line by line	33
8	Housing models and worked exercises	34
8.1	A constant baseline	34
8.2	Linear regression with median income	34

8.3	Plot the line over the test examples	35
8.4	Linear regression with all prepared features	35
8.5	Predicted values against recorded values	36
8.6	Reading the linear coefficients	36
8.7	A random-forest preview	37
8.8	Read the complete comparison	37
8.9	Housing Exercise 1: income plus latitude	38
8.10	Housing Exercise 2: remove the three ratios	38
8.11	Housing Exercise 3: change a district's income input	39
9	Titanic classification: from a score to a decision	39
9.1	Return to the prepared passenger data	39
9.2	The majority-class baseline	39
9.3	What logistic regression computes	40
9.4	Fit the classifier and make predictions	40
9.5	How KNN and a decision tree use these inputs	41
10	The confusion matrix and classification metrics	41
10.1	Start with the actual counts	41
10.2	Accuracy: the overall fraction correct	42
10.3	Precision: reliability among positive predictions	43
10.4	Recall: coverage of the actual positive cases	43
10.5	F1: combining precision and recall	43
10.6	Reading the classification report	43
10.7	Interpreting the logistic coefficients	44
10.8	The model-comparison loop	45
10.9	What would justify choosing one model?	46
11	Overfitting, new passengers, and threshold experiments	46
11.1	Compare training and test performance	46
11.2	Why even an unrestricted model may miss training examples	47
11.3	Prepare two fictional passengers	47
11.4	Exercise 1: compare four neighbour counts	48
11.5	Exercise 2: add fare per person	48
11.6	Exercise 3: raise the decision threshold	49
12	A complete training-only preparation example	50
12.1	Why this companion example is included	50

12.2 Load, construct fixed row features, and split	50
12.3 Describe the preparation steps	51
12.4 Fit the complete sequence and evaluate it	51
12.5 Apply the same principle to housing	52
13 The complete recipe and what each stage means	52
13.1 Read the short slide recipe with its assumptions	52
13.2 A practical sequence for an independent project	53
14 Common problems and how to understand them	53
14.1 A missing file or failed data download	53
14.2 An encoding cell fails after it previously worked	53
14.3 A new input has the wrong columns	53
14.4 The numbers differ from the slides	54
14.5 Other useful checks	54
14.6 A simple recovery procedure	54
15 Revision questions and worked answers	55
15.1 Questions to try before reading the answers	55
15.2 Worked answers	55
15.3 Questions beginners commonly ask	56
15.4 Before Day 3	56
16 Glossary, coverage guide, and sources	57
16.1 Day 2 glossary	57
16.2 Coverage of the supplied files	59
16.3 Important clarifications carried into the explanations	59
16.4 Source files and computational verification	59
16.5 Dataset locations and supplementary documentation	60

1 Starting Day 2

Material covered: Slides 1–3; introductions to all three notebooks.

1.1 The question for today

On Day 1, you trained a classifier using four measurements from each iris flower. The dataset was already organised and ready to use. Today you will spend more time examining the information that enters a model: its columns, units, missing entries, distributions, and relationships.

A model receives a numerical representation of examples. If that representation contains wrong values, exposes an answer that should be hidden, or changes between training and prediction, the result becomes difficult to trust. Data preparation and evaluation are therefore part of the learning problem itself.

There are two prediction tasks today. For the Titanic data, the target is a category: whether a passenger survived. For the housing data, the target is a numerical quantity: the median house value in a district. These let you practise classification and regression with different evaluation measures.

The rainfall question in the recap has the same structure. Predicting tomorrow’s rainfall in millimetres is regression. Predicting whether rain occurs is classification. State the exact question before choosing the model.

1.2 The three notebooks and reading order

Notebook	Main task	Where it fits
1: NumPy and Pandas	Arrays, tables, selection, filtering, summaries, and exercises	The data tools used throughout the other notebooks.
2: Titanic	EDA, preparation, classification, metrics, and new predictions	Begin with exploration and preparation; return to classification after the regression lesson if following the slides.
3: Housing	EDA, preparation, linear regression, metrics, and a random-forest preview	The regression block in the slides.

These notes follow the slide sequence: tools, Titanic exploration and preparation, housing regression, then Titanic classification. Every practical section names its matching notebook part. Repeated operations are explained fully at their first substantial use and then connected to the new task.

The three-hour session allows about 10 minutes for the recap, 30 minutes for data tools, 30 minutes for EDA, 25 minutes for preparation, a 10-minute break, 35 minutes for regression, 35 minutes for classification, and five minutes for the closing review. This document provides additional detail for study after class.

1.3 Before running the examples

Open each notebook as an editable copy and run its cells in order. Each notebook can be started in its own runtime. Variables from Notebook 1 do not automatically appear in Notebook 2. Each has its own imports and data-loading steps.

These notebooks need a dataset download. Seaborn’s Titanic loader normally downloads the CSV

when it is absent from its cache; the housing notebook reads a CSV from the URL written in its first code cell. A network error at that stage is a data-access problem. Check the connection and source address before debugging later model cells.

The code shown here preserves the supplied computations. Some long statements are displayed across lines, and decorative symbols in comments or printed labels are simplified for LaTeX. Empty exercise cells are represented by the exercise explanations and solutions.

1.4 How to read the checked results

The main numerical results below reproduce the supplied notebooks with their original settings. There are a few shortcuts in their preparation order: missing-value statistics are calculated before splitting, and category columns are constructed using the whole dataset. These notes explain those operations as written and then give a training-only preparation pattern to use in a stricter experiment.

Use the supplied results as teaching demonstrations. Repeatedly comparing models and features on the same test set also makes that set part of development. A final project needs a separate evaluation strategy, including validation for choices and an untouched test set for the final report.

2 NumPy: calculations on arrays

Material covered: Slides 4–6; Notebook 1, Part 1.

2.1 Importing the library

```
import numpy as np
print("NumPy version:", np.__version__)
```

`import numpy as np` loads NumPy and gives it the conventional short name `np`. An alias saves typing; it does not create a different library. `np.__version__` displays the installed version. Recording versions helps explain differences between computing environments.

A NumPy array stores an organised collection of values. The arrays here contain numbers. Each array has a shape and a data type. A common data type lets numerical operations use efficient routines, and the shape tells those routines how the values are arranged.

2.2 Doubling prices and understanding vectorisation

```
prices_list = [100, 250, 80]
doubled_list = [p * 2 for p in prices_list]      # needs a loop
print("List:", doubled_list)

prices = np.array([100, 250, 80])
doubled = prices * 2                             # no loop!
print("NumPy:", doubled)
```

The list comprehension visits each value in `prices_list`, multiplies it by two, and builds another list. Its output is `[200, 500, 160]`. The name `p` temporarily refers to one price during each iteration.

`np.array([100, 250, 80])` creates an array from the list. Multiplying that array by two applies the multiplication to each entry, giving the same numerical result. NumPy displays arrays with a different punctuation style, so the printed output is `[200 500 160]`.

This style of whole-array expression is called **vectorisation**. The program states an operation on a collection, and the library carries out the repeated numerical work in its implementation. Computation still takes place for the individual elements. The advantage is that much of the repetition can run in efficient compiled routines.

A plain Python list has different multiplication behaviour. For example, `[100, 250, 80] * 2` repeats the list to produce six items. Recognising the object type helps you predict what an operator will do.

2.3 The speed experiment

```
import time

big_list = list(range(1_000_000))
big_array = np.arange(1_000_000)

start = time.time()
result_list = [x + 1 for x in big_list]
list_time = time.time() - start

start = time.time()
result_array = big_array + 1
array_time = time.time() - start

print(f"Python list: {list_time*1000:.1f} ms")
print(f"NumPy array: {array_time*1000:.1f} ms")
print(f"NumPy was about {list_time / array_time:.0f}x faster")
```

`range(1_000_000)` represents integers from zero through 999,999. Underscores make a long integer easier to read and do not change its value. `list(...)` materialises the sequence as a list. `np.arange(1_000_000)` creates the corresponding NumPy array.

The program records the current time, performs an addition operation, then subtracts the start time from the new time. The elapsed time is in seconds. Multiplying by 1,000 converts it to milliseconds. Finally, dividing the list time by the array time estimates a speed ratio for these particular runs.

Array construction happens before the timer starts. The experiment therefore times the addition step, including creation of its result, rather than the entire data-loading process. It is a simple classroom timing exercise. Hardware, operating-system activity, memory, array size, and measurement overhead can change the result.

In the verification run for these notes, the list addition took about 34.7 ms and the array addition about 4.2 ms, giving roughly eight times the speed. Your result can be different. There is no promised 10–50 times multiplier. For more careful timing, repeat the measurement and use a timing utility designed for short code fragments.

2.4 Creating several kinds of arrays

```
a = np.array([3, 7, 1, 9])      # from a list
z = np.zeros((2, 3))           # 2 rows, 3 columns of zeros
o = np.ones(4)                 # four ones
r = np.arange(0, 10, 2)        # start, stop (not included), step
l = np.linspace(0, 1, 5)       # 5 evenly spaced numbers from 0 to 1

print("a =", a)
print("z =\n", z)
print("o =", o)
print("r =", r)
print("l =", l)
```

The five constructors demonstrate different ways to describe numerical data:

np.array([3, 7, 1, 9])

Creates an array containing those four values in that order.

np.zeros((2, 3))

Creates two rows and three columns of zeros. The pair (2, 3) is the requested shape.

np.ones(4)

Creates a one-dimensional array containing four ones.

np.arange(0, 10, 2)

Starts at zero, advances by two, and stops before ten. The result is [0, 2, 4, 6, 8].

np.linspace(0, 1, 5)

Places five evenly spaced values between zero and one, including both endpoints by default: [0, 0.25, 0.5, 0.75, 1].

arange specifies a step size; **linspace** specifies a number of values. Use the description that matches the problem. For example, plotting a smooth fitted line often starts with a chosen number of evenly spaced inputs.

The zeros and ones print with decimal points because their default type is floating point. The small array **a** is inferred as an integer array. The `\n` inside a quoted print string inserts a line break in the displayed output.

2.5 Shape, rows, columns, and indexing

```
g = np.array([[1, 2, 3],
              [4, 5, 6]])

print("shape:", g.shape)      # (2, 3)
print("g[1, 2] =", g[1, 2])  # row 1, column 2 -> 6
print("first row:", g[0])     # [1 2 3]
print("last column:", g[:, -1]) # every row, last column -> [3 6]
```

The array has two rows and three columns. Its shape is (2, 3). For a two-dimensional array, the first dimension counts rows and the second counts columns. A one-dimensional array of four values has shape (4,); it does not yet have separate row and column dimensions.

Indices begin at zero. The expression `g[1, 2]` selects row index 1 and column index 2, which holds 6. Selecting `g[0]` returns the entire first row, [1, 2, 3]. In `g[:, -1]`, the colon means every row,

and `-1` means the final column, giving `[3, 6]`.

Think of the comma as separating the choice made along each dimension. Many later errors occur because a model receives an input with the wrong shape, such as one flat list where a table of sample rows is expected. Inspecting `.shape` is often an efficient first check.

2.6 Arithmetic and broadcasting

```
temps_c = np.array([30, 25, 35])
temps_f = temps_c * 9/5 + 32
print(temps_f)          # [86. 77. 95.]
```

The Celsius-to-Fahrenheit formula is $F = 9C/5 + 32$. Applying it to the array gives `[86., 77., 95.]`. Each Celsius value is multiplied by $9/5$ and then increased by 32.

A scalar such as 32 can be combined with every element of the array without creating a separate written list of 32s. This is a simple example of **broadcasting**. NumPy has shape rules governing which arrays can be combined in this way. Compatible shapes allow the calculation; incompatible shapes produce an error.

Ordinary `*` between two matching arrays performs element-by-element multiplication. Matrix multiplication uses a different operation, commonly `@`. Today the notebook uses elementwise arithmetic.

2.7 Mean, maximum, and standard deviation

```
print("mean:", a.mean())
print("max:", a.max())
print("std:", round(a.std(), 2))

print("Which are > 5?", a > 5)      # True/False for each element
print("Keep only > 5:", a[a > 5])   # [7 9]
```

For $a = [3, 7, 1, 9]$, the mean is $(3 + 7 + 1 + 9)/4 = 5$. The maximum is 9. The **standard deviation** describes the spread of the values around their mean.

With NumPy's default population-style calculation,

$$\sigma = \sqrt{\frac{(3-5)^2 + (7-5)^2 + (1-5)^2 + (9-5)^2}{4}} = \sqrt{10} \approx 3.16.$$

The squared differences are 4, 4, 16, and 16. Their average is 10, and its square root is about 3.16. The standard deviation has the same units as the original values.

NumPy's default `std()` divides the sum of squared deviations by n . Pandas' usual `Series.std()` default uses $n - 1$, a sample-standard-deviation convention. These defaults can explain differing outputs from similarly named operations. State the convention when it matters to your calculation.

2.8 Boolean filtering

The expression `a > 5` compares each entry with five and gives

```
[False, True, False, True]
```

This Boolean array is a **mask**. Using it inside `a[a > 5]` retains the positions marked true, so the result contains 7 and 9. The condition describes which items to keep.

A threshold exactly equal to five would be excluded by `> 5`. Using `>= 5` includes equality. The same masking idea will select rows from the Titanic table.

2.9 The exam-score example and the meaning of axis

```
scores = np.array([
    [72, 65, 80], # Ada
    [55, 70, 62], # Bola
    [90, 85, 88], # Chidi
    [48, 52, 60], # Dayo
])
students = ["Ada", "Bola", "Chidi", "Dayo"]

print("Average per student:", scores.mean(axis=1).round(1))
print("Average per subject:", scores.mean(axis=0).round(1))
print("Best student:", students[scores.mean(axis=1).argmax()])
```

The score array has shape `(4, 3)`: four students and three subjects. Each row is one student's marks in Maths, Physics, and Chemistry. The separate list `students` stores the names in the same row order.

An **axis** identifies a dimension being reduced by an operation. For this table, `axis=1` reduces the columns within each row. It gives one average per student. `axis=0` reduces the rows within each column, giving one average per subject.

Student	Maths	Physics	Chemistry	Row mean
Ada	72	65	80	72.3
Bola	55	70	62	62.3
Chidi	90	85	88	87.7
Dayo	48	52	60	53.3
Column mean	66.25	68.0	72.5	

For Ada, $(72 + 65 + 80)/3 = 72.333\dots$. For Maths, $(72 + 55 + 90 + 48)/4 = 66.25$. The notebook rounds display values to one decimal place. The Maths mean displays as 66.2 in the checked environment, consistent with NumPy's rounding convention for this halfway value.

`scores.mean(axis=1).argmax()` finds the position of the largest student average. It returns index 2. Using that index in `students` retrieves Chidi. `argmax` returns a position, while `max` returns the maximum value itself.

Pause and try. What shape should the result have after taking the mean along axis 0 of a (4,3) array?

Answer: Shape (3,), because one value remains for each of the three subjects.

2.10 Notebook 1, Exercise 1: complete solution

```
# Exercise 1
voltages = np.array([3.3, 5.0, 12.0, 9.0, 1.8])
print("How many:", voltages.shape[0])
print("Average:", voltages.mean().round(2))
print("Above 4 V:", voltages[voltages > 4])
print("Scores out of 20:\n", scores / 100 * 20)
```

The voltage array contains five values, so `voltages.shape[0]` returns 5. Their total is 31.1 V, and the average is $31.1/5 = 6.22$ V. The mask retains 5.0, 12.0, and 9.0 because these are above 4 V.

To convert a score out of 100 into a score out of 20, divide by 100 to obtain its fraction of the maximum, then multiply by 20. For example, $72/100 \times 20 = 14.4$. The full converted array is

$$\begin{bmatrix} 14.4 & 13.0 & 16.0 \\ 11.0 & 14.0 & 12.4 \\ 18.0 & 17.0 & 17.6 \\ 9.6 & 10.4 & 12.0 \end{bmatrix}.$$

The array expression preserves the row and column arrangement, so each converted score still belongs to the same student and subject.

3 Pandas: reading and working with tables

Material covered: Slides 7–8; Notebook 1, Part 2.

3.1 DataFrames, Series, and the index

```
import pandas as pd

students_df = pd.DataFrame({
    "name": ["Ada", "Bola", "Chidi", "Dayo"],
    "dept": ["EEE", "MCT", "EEE", "BME"],
    "level": [200, 200, 200, 200],
    "gpa": [4.2, 3.6, 4.8, 3.1],
})
students_df
```

A **DataFrame** is a table with named columns and an index identifying rows. A **Series** is a one-dimensional labelled collection, such as one column extracted from a DataFrame.

The dictionary keys become column names: name, department, level, and GPA. Each corresponding list provides four values. Equal list lengths ensure that each row can be constructed consistently. The default row index is 0 through 3.

An index label identifies a row within the table. It can remain attached to that row after sorting or filtering, so it need not match the row's new visible position. Later, `iloc` selects by position, while `loc` can select by index label or a Boolean condition.

3.2 Loading the Titanic table

```
import seaborn as sns

df = sns.load_dataset("titanic")
df.head()           # first 5 rows
```

Seaborn provides a convenient loader for several example datasets. The Titanic table is fetched from its dataset collection when needed and can be cached locally. The table has one row per recorded passenger and initially contains 891 rows and 15 columns.

A CSV is a text file storing table-like data, commonly with comma-separated fields and a header row. For your own downloaded file, a common starting point is:

```
df = pd.read_csv("your_file.csv")
```

`df.head()` displays the first five rows. This preview helps identify column names and representative values, but the first five rows may not represent the full distribution. Follow it with structural checks and summaries.

3.3 The first inspection commands

```
print("Shape (rows, columns):", df.shape)
```

```
df.info()           # column names, types, and how many non-missing values
```

```
df.describe()       # statistics for the number columns
```

```
df.isna().sum()     # how many missing values in each column
```

`shape` returns the number of rows and columns, here (891, 15). It is an attribute, so it is accessed without parentheses. `info()` is a method that prints column names, non-missing counts, data types, and memory information.

The `age` column has 714 non-missing entries. Subtracting from 891 gives 177 missing ages. `int64` describes an integer representation, `float64` describes floating-point numbers, `bool` describes Boolean values, and `category` describes categorical data. Text often appears as `object` in this environment. Type representations can differ between library versions.

`describe()` summarises numerical columns by default. Its rows have distinct meanings:

Count

Number of non-missing values included in the column summary.

Mean

Arithmetic average of those values.

Std Sample standard deviation under Pandas' default convention.

Min and max

Smallest and largest observed values.

25 percent, 50 percent, and 75 percent

Quartiles. The 50-percent value is the median; the other two mark the lower and upper quarters of the ordered distribution, according to the quantile calculation.

For age, the mean is about 29.7 years, median 28, and maximum 80. For fare, the mean is about 32.2, median 14.4542, and maximum 512.3292. A few large fares raise the mean substantially above the median.

`isna()` produces a table of Boolean missing-value indicators. Summing it counts true entries column by column, because true behaves like one and false like zero in this calculation. The result identifies 177 missing ages, 688 missing decks, two missing embarkation codes, and two missing embarkation-town names.

A missing value is an absence of recorded information. It is not automatically a zero, an average, or evidence that an event did not happen. Deciding how to handle it requires understanding the column and the task.

3.4 Selecting one column or several columns

```
df["age"].head()
```

```
df[["sex", "age", "survived"]].head()
```

`df["age"]` selects a Series. `df[["sex", "age", "survived"]]` selects a DataFrame using a list of column names. The inner brackets create that list; the outer brackets apply the selection to the table.

Column names must match exactly, including spelling and capitalisation. The order of names in the selection determines their displayed order. Many model interfaces expect a two-dimensional table, so the distinction between a Series and a one-column DataFrame matters even when they contain the same numbers.

3.5 Filtering passenger rows

```
kids = df[df["age"] < 18]
print("Number of children:", len(kids))

women_first_class = df[(df["sex"] == "female") & (df["pclass"] == 1)]
print("Women in 1st class:", len(women_first_class))
```

`df["age"] < 18` tests the age in every row. The outer selection keeps the true rows, creating the table `kids`. The verified count is 113 passengers with known age below 18. Rows with missing ages do not pass this comparison.

The next filter combines two requirements: the recorded sex is female and ticket class equals 1. Each comparison has its own parentheses, and `&` combines the resulting Boolean masks. The selected group contains 94 passengers.

For Series-wise conditions, use `&` for logical and, `|` for logical or, and `~` to invert a Boolean mask.

Python's ordinary `and` and `or` are not the corresponding elementwise operators for a whole Series. Parentheses also prevent operator-precedence mistakes.

The slide's 59-percent child survival statistic uses age below 16, while this filtering exercise uses age below 18. The verified under-16 group contains 83 passengers with a survival rate of about 59.0 percent. Both calculations can be correct with their stated definitions.

3.6 Adding family size

```
df["family_size"] = df["sibsp"] + df["parch"] + 1
df[["sibsp", "parch", "family_size"]].head()
```

`sibsp` records siblings and spouses aboard. `parch` records parents and children aboard. Adding these counts and one for the passenger produces the notebook's family-size feature. A passenger with one sibling or spouse and no parents or children receives $1 + 0 + 1 = 2$.

The right-hand side calculates a value for every row. Assignment stores the result in a new column. This is an example of feature engineering: constructing a potentially useful description from existing information. It reflects the relatives represented by these fields; it may not describe the passenger's complete travelling party.

The table now has one additional column. A later call to `df.shape` can therefore report 16 columns. The earlier 15-column result described the table before this assignment.

3.7 Group and summarise

```
df.groupby("sex")["survived"].mean().round(2)
```

```
df.groupby("pclass")["survived"].mean().round(2)
```

Read the first expression as: group rows by recorded sex, select survival outcomes within each group, compute their mean, and round the displayed result to two decimal places.

The survival target is coded zero or one. If a group contains n passengers and s survivors, its target values sum to s . Therefore its mean is s/n , the survival proportion. This explains why averaging a binary indicator produces a rate.

The observed rates are about 0.742 for female passengers and 0.189 for male passengers. By ticket class, they are about 0.630, 0.473, and 0.242 for classes 1, 2, and 3. These summarise the records in this dataset. They show associations and do not isolate a causal effect of any one characteristic.

3.8 Counting categories and sorting rows

```
print(df["embarked"].value_counts())
print()
print(df.sort_values("fare", ascending=False)[["pclass", "sex", "age", "fare"]].head())
```

`value_counts()` counts the occurrences of each category. It reports 644 Southampton records (S), 168 Cherbourg records (C), and 77 Queenstown records (Q). These total 889 because the two

missing codes are excluded by default. To include missing entries in that particular count, use `dropna=False`.

`sort_values("fare", ascending=False)` orders rows from highest to lowest fare. Selecting a subset of columns makes the output easier to inspect, and `head()` limits the display to five rows. Several passengers share the highest recorded fare, 512.3292. The remaining row-index labels identify their original rows; they are not new positions in the sorted output.

Sorting returns a sorted result unless you explicitly assign it back or request an in-place operation where supported. The notebook uses it for display without replacing the entire table's order.

3.9 Notebook 1, Exercise 2: full solution

```
# Exercise 2
print("3rd class passengers:", len(df[df["pclass"] == 3]))
print()
print("Average age by survival (0 = died, 1 = survived):")
print(df.groupby("survived")["age"].mean().round(1))
print()
print("Survival rate by port:")
print(df.groupby("embarked")["survived"].mean().round(2))
print()
df["fare_per_person"] = df["fare"] / df["family_size"]
print(df.sort_values("fare_per_person", ascending=False)[["pclass", "fare", "family_size", "fare_per_person"]].head())
```

Task 1: count third-class passengers. The comparison creates a Boolean mask for `pclass == 3`. Selecting those rows and applying `len` gives 491.

Task 2: mean age by outcome. Group by survival status and average the age within each group. The observed mean age is 30.6 years among non-survivors and 28.3 among survivors, rounded to one decimal place. Missing ages are omitted from these averages, so the age summaries cover only known-age passengers in each outcome group.

Task 3: survival by embarkation port. Group by port and average the binary target. The rounded rates are 0.55 for C, 0.39 for Q, and 0.34 for S. Differences between port groups can reflect differences in passenger composition, such as ticket class, as well as other factors.

Task 4: construct fare per person. Divide each row's fare by its engineered family size, then sort by the new column. The five highest displayed values are approximately 512.3292, 512.3292, 256.1646, 227.5250, and 227.5250. This constructed ratio is a modelling feature; the data does not establish that it exactly measures each individual's payment.

4 Exploratory data analysis with the Titanic

Material covered: Slides 9–13; Notebook 2, Parts 1–2.

4.1 The six questions behind EDA

Exploratory data analysis, or EDA, is the process of examining a dataset before making detailed modelling decisions. It combines numerical summaries, plots, and knowledge of how the data was produced. The six questions in the slides form a practical starting sequence:

1. How many examples and columns are present?

2. What does each column mean, and what is its data type?
3. Which values are missing?
4. How are numerical values distributed?
5. Which variables appear related?
6. Which records or patterns need investigation?

An unusual observation can be an error or a valid rare event. EDA helps you decide what to inspect next. It does not justify automatically deleting observations simply because they lie far from the centre of a plot.

4.2 Read the passenger fields before plotting

Field	Meaning in the modelling subset
survived	Target: 0 means did not survive, 1 means survived.
pclass	Ticket-class code: 1, 2, or 3.
sex	Recorded category in the source table.
age	Age in years, with some missing values.
sibsp	Number of siblings and spouses aboard.
parch	Number of parents and children aboard.
fare	Recorded ticket fare.
embarked	Port code: C, Q, or S.

The 891 records form the teaching dataset used here. They are not a record of every person aboard the ship. Always state which dataset a reported rate describes.

4.3 Load a fresh copy and inspect it

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
df = sns.load_dataset("titanic")
print(df.shape)
df.head()
```

The imports supply arrays, tables, and plotting tools. `sns.set_theme(style="whitegrid")` sets the visual style, including a light grid. It changes plot appearance rather than the data. Loading the Titanic here creates a fresh table for Notebook 2, independent of new columns created in Notebook 1.

```
df.info()
```

```
missing = df.isna().sum()
missing[missing > 0]
```

```
print(f"Survival rate overall: {df['survived'].mean():.1%}")
```

The missing-value count is first stored in a Series called `missing`. The expression `missing[missing > 0]` filters this Series to show only columns with gaps. Here it reports age, embarked, deck, and embarkation town.

The overall target mean is $342/891 \approx 0.3838$. The format `.1%` converts that proportion to a percentage with one decimal place, so it prints 38.4 percent. There are 549 non-survivors and 342 survivors in this table.

4.4 A count plot: how many examples are in each category?

```
sns.countplot(data=df, x="survived")
plt.title("0 = died, 1 = survived")
plt.show()
```

A count plot places one bar at each category and makes its height equal to the number of rows in that category. The horizontal axis has outcomes 0 and 1. The taller bar at zero represents the 549 non-survivors; the shorter bar at one represents the 342 survivors.

This immediately shows unequal class frequencies. A classifier that always predicts zero can already achieve a substantial accuracy. That observation motivates the baseline calculation later.

4.5 Using hue to split the counts

```
sns.countplot(data=df, x="sex", hue="survived")
plt.title("Survival by sex")
plt.show()

print(df.groupby("sex")["survived"].mean().round(3))
```

The main categories along the horizontal axis are the recorded sex categories. The `hue="survived"` argument subdivides each category by outcome using different colours. Consult the legend to identify which colour refers to survival.

The plot displays counts, so group sizes affect bar heights. The accompanying `groupby` result expresses survival within each group as a proportion. A group can contain more total people while having a smaller survival rate. Both counts and denominators matter.

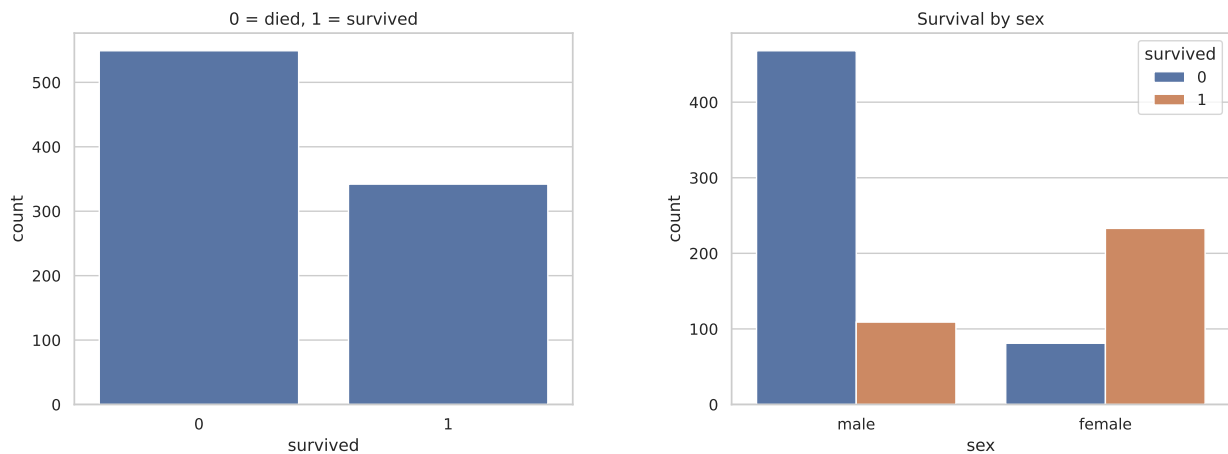


Figure 1: Overall outcome counts and counts separated by recorded sex, reproduced from Notebook 2.

4.6 A bar plot of a mean: survival by ticket class

```
sns.barplot(data=df, x="pclass", y="survived")
plt.title("Survival rate by ticket class")
plt.ylabel("survival rate")
plt.show()

print(df.groupby("pclass")["survived"].mean().round(3))
```

Here the vertical variable is the binary survival target. Seaborn's bar plot uses its mean by default, so the heights show survival rates within each ticket class. The y-axis is a proportion, with values from zero to one relevant to this target.

The small error bars in this Seaborn version show bootstrap 95-percent confidence intervals for the estimated group means under the plotting procedure. Bootstrapping repeatedly resamples a group's observations and recomputes the estimate. These bars describe uncertainty under that procedure, not the range of individual zero-or-one outcomes. The notebook does not fix the bootstrap seed, so small visual differences can occur between runs.

Class 1 has an observed survival rate near 63.0 percent, class 2 near 47.3 percent, and class 3 near 24.2 percent. Ticket class appears useful for predicting the outcome, but the plot alone does not establish why these differences occurred.

4.7 A histogram: the distribution of ages

```
sns.histplot(data=df, x="age", hue="survived", bins=30, multiple="stack")
plt.title("Age distribution, coloured by survival")
plt.show()
```

A histogram divides a numerical range into intervals called **bins**. Each bar counts observations whose values fall in that interval. `bins=30` requests 30 bins. The `hue` separates the outcomes, and `multiple="stack"` places their contributions on top of each other within a bin.

The total height at a given age interval indicates how many passengers with known age fall there. The colour segments indicate their outcomes. Missing ages do not enter the histogram. Its total

therefore corresponds to 714 known-age records rather than all 891 passengers.

A stacked count histogram does not directly show the probability of survival at each age. To estimate a survival rate within an age range, divide survivors in that range by all known-age passengers in the same range. Very narrow bins can make a plot noisy; very wide bins can hide local structure.

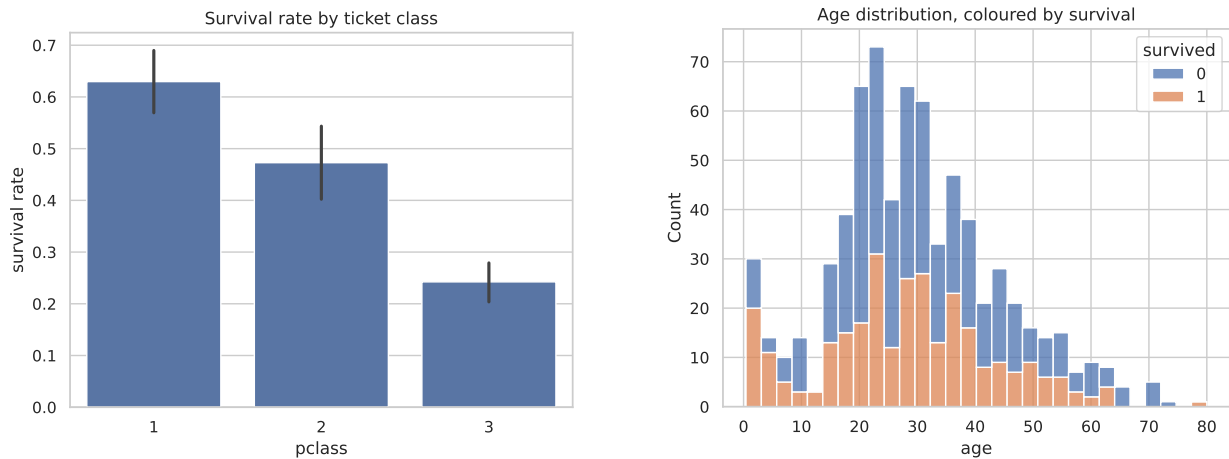


Figure 2: Survival-rate estimates by class and a stacked count histogram of known ages.

4.8 A box plot: fare spread and unusual values

```
sns.boxplot(data=df, x="pclass", y="fare")
plt.title("Fare by ticket class")
plt.show()

print("Fare mean:", round(df["fare"].mean(), 1), "| median:", df["fare"].median(), "| max:", df[
    "fare"].max())
```

A box plot summarises a distribution using quartiles. The box extends from the first quartile Q_1 to the third quartile Q_3 , covering the middle 50 percent of values. The line inside the box marks the median. The **interquartile range**, or IQR, is $Q_3 - Q_1$.

Under the usual box-plot rule used here, whiskers extend to the most extreme observed values within 1.5 IQR of the box limits. Observations beyond those limits are plotted individually. They are called outliers under this plotting rule. That label identifies unusual values; it does not prove that their records are erroneous.

The fare data has a long upper tail. Its mean is about 32.2, its median about 14.45, and its maximum about 512.33. First-class fares generally occupy a higher range than third-class fares. Some high fares may reflect valid ticket arrangements, so they need interpretation before any cleaning decision.

4.9 A correlation heatmap

```
num_cols = ["survived", "pclass", "age", "sibsp", "parch", "fare"]
sns.heatmap(df[num_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation between number columns")
plt.show()
```

The list `num_cols` explicitly chooses the six numerical columns to compare. The method `corr()` calculates their pairwise Pearson correlations using available paired observations. Pearson correlation measures linear association on a scale from -1 to $+1$.

A value near $+1$ indicates a strong positive linear association: larger values of one variable tend to occur with larger values of the other. A value near -1 indicates a strong negative linear association. A value near zero indicates weak linear association; a curved relationship may still exist.

The heatmap displays that matrix with colours. `annot=True` prints each value in its cell, `fmt=".2f"` requests two decimal places, and `vmin=-1`, `vmax=1` fixes the colour scale. `cmap="coolwarm"` names the palette. The diagonal entries are one because a varying column is perfectly correlated with itself.

Ticket-class code and fare have correlation about -0.55 . This is sensible given the coding: first class uses the smaller number 1 and tends to have higher fares. Survival and ticket-class code have correlation about -0.34 . A negative value reflects the direction of that coded association.

The recorded sex column is absent from this numerical heatmap because it still contains text. Absence from the heatmap says nothing about whether it is informative. Its relationship was examined through grouped rates and count plots.

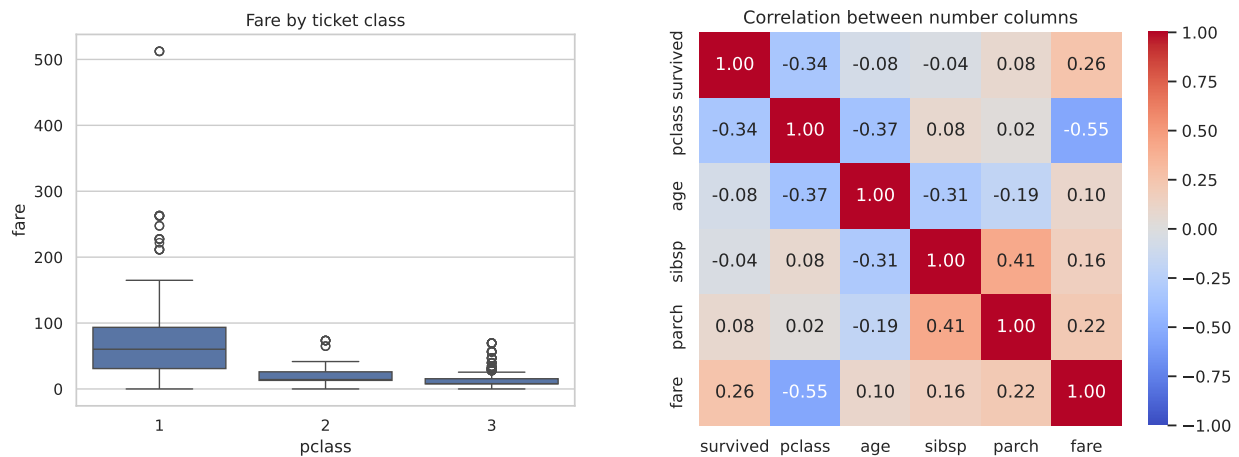


Figure 3: Fare distributions and pairwise numerical correlations. Read the axis labels and colour scale alongside the values.

4.10 Choosing the plot for the question

Question	Plot	What to inspect
How many records are in each category?	Count plot	Counts and class balance.
What is a group's mean outcome?	Bar plot	Group means, denominators, and uncertainty.
How is one number distributed?	Histogram	Concentration, spread, skew, gaps, and tails.
How do numerical distributions compare?	Box plot	Medians, quartiles, whiskers, and unusual points.
How do two numerical features relate?	Scatter plot	Direction, curvature, overlap, and clusters.
How are multiple numerical columns linearly related?	Heatmap of correlations	Pairwise correlation signs and magnitudes.

The scatter plot appears extensively in the housing notebook. Each point represents one row positioned by two numerical columns. A third variable can be represented by colour.

Many Seaborn functions accept named columns through **data**, **x**, **y**, and **hue**, where relevant. Their signatures are not identical. A heatmap usually receives an already computed two-dimensional matrix, as it does here.

4.11 Answering the EDA discussion questions

Female passengers and first-class passengers have higher survival rates in the reported one-variable group summaries. The lowest ticket-class rate belongs to class 3, and the male group rate is lower than the female group rate. A claim about a combined subgroup, such as third-class men, requires grouping jointly by both fields.

The recorded sex field shows a strong association with the target and is a sensible candidate when asked which single column to investigate first. To establish which single feature gives the strongest predictive performance under a specified model, compare candidates using an appropriate validation procedure.

EDA can suggest explanations, but association alone does not establish a cause. The class, fare, age, and sex patterns reflect overlapping characteristics and a historical context. The notebook is demonstrating statistical analysis and prediction from records.

5 Cleaning, encoding, features, and splitting

Material covered: Slides 14–19; Notebook 2, Parts 3–6.

5.1 First remove information that gives away the answer

```
pd.crosstab(df["survived"], df["alive"])
```

A **cross-tabulation** counts combinations of values from two columns. Here it produces:

	Alive: no	Alive: yes
survived = 0	549	0
survived = 1	0	342

The alignment is exact: **alive** states the same outcome as **survived** in words. Supplying it as a predictor would expose the target. At the intended prediction time, that outcome would be unknown. This is direct **target leakage**.

Other columns, such as the written ticket class or embarkation-town name, repeat information already present in another form. Columns such as **who** and **adult_male** derive categories from age and sex. Redundancy and leakage are separate issues: redundant information can be available at prediction time, while a leaked outcome violates the intended information boundary. The notebook uses a small explicit subset to keep the exercise manageable.

```
data = df[["survived", "pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]].copy()
data.isna().sum()
```

The list selects the target plus seven original input fields. `copy()` creates a separate table for the preparation steps. Keeping the original `df` available lets you return to its raw columns for checking.

5.2 Missing values: what are the options?

The three options in the slides are to drop a column, drop affected rows, or fill the gaps. Filling a missing value is called **imputation**. The right choice depends on the amount and pattern of missingness, the value of the remaining information, and the model being used.

Dropping every row with a missing age would remove 177 passengers, about 20 percent of this table. That also discards the known class, fare, and outcome for those passengers. Dropping deck is a reasonable simplification for this exercise because 688 of its 891 values are missing. A high missing fraction alone is not a universal rule that a column has no useful information.

A numerical median is a simple imputation choice. The median is the middle of the ordered values and is less affected by extreme magnitudes than the mean. A **mode** is a most frequently occurring value and is a common categorical imputation choice.

Some estimators can handle missing values directly, while others require explicit preparation. The models and workflow used here use complete numerical inputs, so we fill the selected columns before fitting them.

5.3 The imputation code in the supplied notebook

```
print("Median age:", data["age"].median())
print("Most common port:", data["embarked"].mode()[0])

data["age"] = data["age"].fillna(data["age"].median())
data["embarked"] = data["embarked"].fillna(data["embarked"].mode()[0])

data.isna().sum().sum() # total missing values left -> should be 0
```

The complete-data median age is 28 and the most common embarkation code is S. `fillna` replaces missing entries and leaves observed entries unchanged. Assigning the result back to the same column stores the completed values.

`mode()` returns a Series because a column can have more than one mode. Selecting `[0]` takes the first result. The final expression first counts missing entries per column and then sums those counts, producing a total of zero for the selected table.

Imputed ages are estimates used to make the input usable; the method has not recovered the passengers' unknown ages. Filling many rows with one number also creates repeated values and can change a distribution. An additional missingness indicator can preserve the information that a value was originally absent.

Preparation-order clarification. This cell calculates its median and mode from all rows before the split. For a strict predictive evaluation, calculate these statistics on training rows only and apply those stored values to both training and test rows. The housing notebook has the same issue with bedroom imputation. A small apparent effect on one score does not make the order a reliable general practice.

5.4 Family size and travelling alone

```
data["family_size"] = data["sibsp"] + data["parch"] + 1
data["is_alone"] = (data["family_size"] == 1).astype(int)

print(data.groupby("is_alone")["survived"].mean().round(3))
```

The family-size calculation is the same as in Notebook 1. The equality comparison checks whether the resulting value is one. `astype(int)` converts true to 1 and false to 0, giving a numerical indicator for travelling without the relatives represented in these fields.

The group mean is about 0.304 for `is_alone=1` and 0.506 for `is_alone=0`. These are survival rates of about 30.4 and 50.6 percent. They describe an association within this dataset.

```
sns.barplot(data=data, x="family_size", y="survived")
plt.title("Survival rate by family size")
plt.show()
```

The bar plot estimates survival rate at each family size. Small families of two to four show relatively high rates in these records. Some large-family categories have few observations, so inspect their counts before drawing a broad conclusion. Uncertainty is particularly relevant for small groups.

Feature engineering can give a model an accessible form of a relationship. It can also add redundant or unstable information. Here family size is exactly related to the existing relative counts, so individual coefficients in a linear model need careful interpretation when all are retained.

5.5 Binary encoding and category codes

```
data["sex"] = data["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(data, columns=["embarked"], drop_first=True, dtype=int)
data.head()
```

The `map` call replaces male with 0 and female with 1 according to the chosen dictionary. These numbers encode categories. They do not express a natural numerical magnitude of the underlying categories.

The port field has three nominal categories with no useful numerical ordering. Assigning arbitrary integers such as 1, 2, and 3 can introduce unintended ordering or distances for models that use those numerical relationships. One-hot encoding uses indicator columns instead.

`pd.get_dummies` replaces the selected categorical column with indicators. `dtype=int` produces integer zeroes and ones. `drop_first=True` removes one indicator to use as a reference category. In this notebook, the retained columns are `embarked_Q` and `embarked_S`; category C is dropped.

Port	embarked_Q	embarked_S
C	0	0
Q	1	0
S	0	1

This corrects the text in the supplied notebook and speaker notes, which describes S as the dropped category. The code and the new-passenger example use the table shown above. Inspect the generated column names whenever the choice of reference matters.

Dropping an indicator avoids an exact dependency among a complete set of indicators and an intercept in a basic linear model. Different estimators and regularisation choices can also work with all indicators. The housing notebook keeps all of its ocean-category indicators.

Ticket class stays as 1, 2, or 3 in this exercise. Its order is meaningful, but a linear model also imposes a particular relationship across steps in that code. One-hot encoding ticket class would permit more flexible category effects. Keeping the original representation here preserves the supplied experiment.

5.6 Why scaling changes the numerical problem

Scaling adjusts the numerical representation of features. In KNN, differences along each feature contribute to a distance. A feature with a large numerical spread can contribute more strongly unless the representation is adjusted appropriately.

Standardisation subtracts a training mean and divides by a training standard deviation:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

If a training age column had mean 30 and standard deviation 10, an age of 50 would become $(50 - 30)/10 = 2$. A value of 20 would become -1 . These transformed values express position relative to the training mean in units of training standard deviation.

For a non-constant feature, the transformed training column has mean approximately zero and standard deviation one under the scaler's convention. The transformed test column generally does not have exactly these values, because its distribution can differ. A constant column is handled

specially by the scaler and cannot be given a nonzero empirical standard deviation through this transformation.

Standardisation does not force a bell-shaped distribution, eliminate outliers, or place every value between zero and one. It also does not convert age and fare into the same physical unit. It creates a dimensionless representation based on each feature's training statistics.

Min-max scaling uses training minimum and maximum:

$$x' = \frac{x - x_{\min, \text{train}}}{x_{\max, \text{train}} - x_{\min, \text{train}}}.$$

For a varying feature, this maps the training range to zero through one. A future value outside that training range can transform outside zero through one unless clipping is explicitly applied.

Scaling is especially useful for distance-based models and many regularised or gradient-based procedures. Logistic regression can run without standardisation, but scale can influence numerical optimisation and the effect of regularisation. Decision trees generally do not need it because their splits depend on ordered values within a feature.

5.7 Separating the target and splitting paired rows

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = data.drop(columns="survived")
y = data["survived"]
print("Features:", list(X.columns))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

print("Train:", X_train.shape, "| Test:", X_test.shape)
```

`data.drop(columns="survived")` constructs the feature table, while `data["survived"]` supplies the target. The feature table has ten columns after adding family features and encoding the ports.

`test_size=0.2` reserves 20 percent of the examples. For 891 rows, the result is 179 test rows and 712 training rows. The function preserves feature–target pairing. `random_state=42` helps repeat the same split with the same data and environment.

`stratify=y` requests approximately matching target proportions across the sets. The test set has 110 non-survivors and 69 survivors. Stratification helps preserve class representation. It does not remove all sampling variation or handle related passengers as groups.

The feature shapes are (712, 10) for training and (179, 10) for testing. The target arrays contain 712 and 179 values respectively. After scaling, `X_train_s` and `X_test_s` are numerical arrays, so the separate `X.columns` list is used later to label coefficient plots and order new inputs.

5.8 Fit the scaler once, then reuse it

`StandardScaler().fit(X_train)` calculates and stores the training means and scale values. Each `transform` call applies that fitted transformation to its supplied table. The test rows do not set their own scale.

Fitting a new scaler on the test set would give a different coordinate system and use information from the evaluation data. New passengers must also use the training-fitted scaler. The saved model and the preprocessing that defines its inputs belong together.

This scaler step follows the correct training-only order. The earlier imputation step still used the full table, so the entire supplied preparation sequence is not free of leakage. Keep those two observations separate when explaining the notebook.

5.9 A preparation order to use in your own projects

Begin by defining permissible predictors and reserving the evaluation data. Learn imputation values, category vocabularies, scaling parameters, and other fitted transformations from training data. Apply that same prepared mapping to validation or test examples. Fit the estimator on the prepared training examples.

A deterministic row-wise calculation, such as adding the two family counts and one, can be applied consistently without learning a statistic from other rows. Transformations that learn something from a collection of rows need the training boundary. Detailed model-guided EDA and feature selection also belong within development data.

Section 12 gives a complete Titanic pipeline implementing the training-only order. It is a companion example for stronger practice. Its outputs should be evaluated as a separate experiment rather than silently substituted for the original notebook's reported scores.

6 The housing dataset: a regression problem

Material covered: Slides 20–21; Notebook 3, Parts 1–3.

6.1 Understand what one row represents

The housing task predicts a district's median house value using district-level information. One row represents a geographical group of households, described as a district in the supplied material. It is not one individual house. The distinction affects the meaning of both features and predictions.

A district's median house value is the middle of its distribution of recorded house values. It is an aggregate target. A prediction for that median is not an appraisal of a particular home at a particular address.

The table comes from the California housing teaching dataset based on 1990 census information. Its historical dollar values provide a regression exercise. It is not a source of current property valuations.

6.2 Load and inspect the source table

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")

url = "https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv"
housing = pd.read_csv(url)
print(housing.shape)
housing.head()
```

`pd.read_csv(url)` reads the CSV at the supplied address into a DataFrame. The resulting shape is (20640, 10). The URL is part of the notebook's data source, so keep it available when documenting or reproducing the experiment.

```
housing.info()
```

```
housing.describe().round(1)
```

```
housing["ocean_proximity"].value_counts()
```

`info()` identifies one incomplete numerical field: `total_bedrooms`, with 20,433 observed values out of 20,640 rows. The remaining 207 entries are missing. `describe().round(1)` displays summaries rounded to one decimal place. It does not round or replace every underlying value in the dataset.

Column	Interpretation
longitude, latitude	Geographical coordinates identifying the district's location.
housing_median_age	Median housing age in the district.
total_rooms	Total rooms across the district's recorded housing.
total_bedrooms	Total bedrooms, with some missing district totals.
population	District population.
households	Number of households.
median_income	Scaled median-income field; the notebook interprets a unit as approximately ten thousand dollars.
median_house_value	Target median house value in dollars.
ocean_proximity	A categorical description of proximity to the ocean or bay.

The ocean field is categorical rather than a measured continuous distance. Its counts are 9,136 for <1H OCEAN, 6,551 for INLAND, 2,658 for NEAR OCEAN, 2,290 for NEAR BAY, and five for ISLAND. The very small island group deserves caution when interpreting a fitted category coefficient.

The income field uses the dataset's scaled representation and includes restricted extremes. Read its units and limits from the dataset description when interpreting a coefficient. In the notebook's convention, 3.5 corresponds approximately to a median income of \$35,000.

6.3 Inspecting the target distribution and its cap

```
sns.histplot(housing["median_house_value"], bins=50)
plt.title("Median house value per district")
plt.show()

print("Districts at the $500,001 cap:", (housing["median_house_value"] >= 500001).sum())
```

The histogram uses 50 bins to display the distribution of median house values. The expression comparing the target with 500001 produces a Boolean Series, and summing it counts the rows at the recorded cap. There are 965 such districts.

The high bar near \$500,001 reflects a recording limit. A district with a higher underlying value can be represented by that same capped target. This is a form of top-coding or censoring of the target information. The table cannot reveal the actual distinctions among all higher-valued districts.

A model may still produce a prediction above \$500,001. Ordinary linear regression has no built-in upper bound. What the cap prevents is direct learning of the true higher targets from these recorded values. Evaluation against the capped labels also measures agreement with the recorded target rather than with the unknown uncapped values.

6.4 A geographical plot from coordinates

```
plt.figure(figsize=(8, 7))
sns.scatterplot(data=housing, x="longitude", y="latitude", hue="median_house_value",
                palette="viridis", s=8, linewidth=0)
plt.title("California districts coloured by house value")
plt.show()
```

Longitude determines horizontal position and latitude vertical position. These two numerical columns place each district on a geographical scatter plot. `hue="median_house_value"` maps

target values to colour, and `palette="viridis"` selects the colour scale.

Small markers, `s=8`, help show many districts. `linewidth=0` removes marker borders that could obscure dense regions. The visible concentration of high recorded values near some coastal areas suggests that location contains useful information.

This is a coordinate plot without a map background or a detailed projection treatment. It supports an initial view of spatial patterns. A random train/test split can place nearby districts in both sets; a project aimed at generalising to a new region may require spatially separated evaluation.

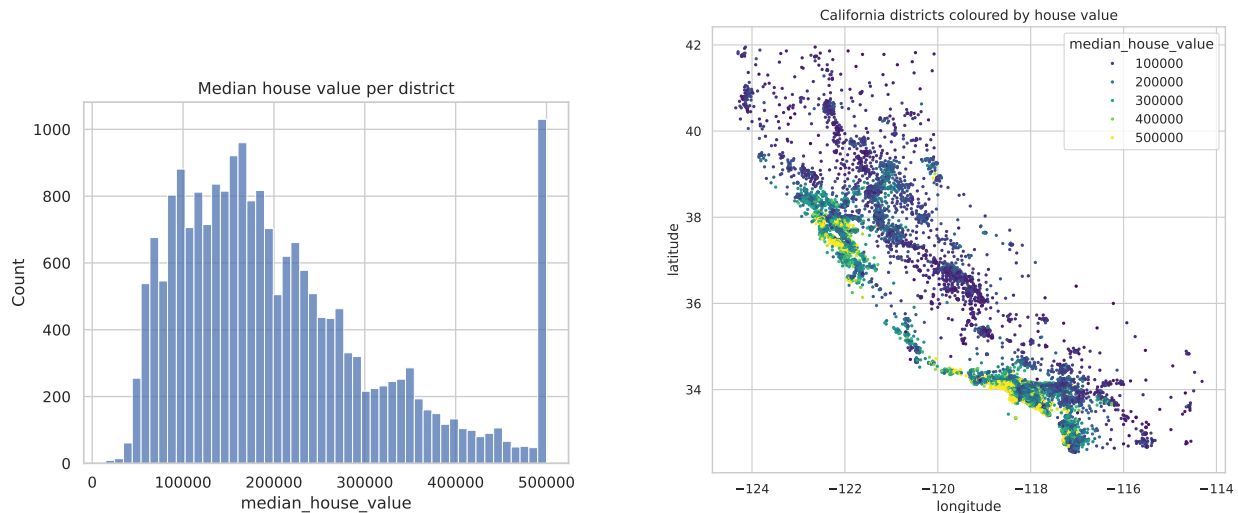


Figure 4: The capped housing target and the district coordinate plot, coloured by recorded median value.

6.5 Looking for numerical relationships

```
corr = housing.select_dtypes("number").corr()["median_house_value"].sort_values(ascending=False)
corr.round(2)
```

`select_dtypes("number")` selects numerical columns before calculating correlations. Selecting the target's column from the correlation matrix gives each numerical field's correlation with the target. Sorting descending places the largest positive values first.

The target's self-correlation is one and provides no predictor information. Among the original numerical inputs, median income has the largest positive Pearson correlation with house value, approximately 0.69. Total rooms has about 0.13, total bedrooms about 0.05, and latitude about -0.14 .

A single Pearson correlation does not measure every kind of predictive relationship. A feature with a weak linear association can still contribute through interactions or nonlinear patterns. A high correlation also does not establish causation.

```
sns.scatterplot(data=housing, x="median_income", y="median_house_value", alpha=0.1, s=10)
plt.title("Income vs house value")
plt.show()
```

The scatter plot puts income on the horizontal axis and house value on the vertical axis. `alpha=0.1` makes each marker partly transparent, so dense areas remain more visible. `s=10` requests small markers.

The rising pattern suggests that income is useful for a first linear model. The horizontal band at the target cap and the spread of prices at similar incomes show that a single line will leave substantial errors.

6.6 Filling the missing bedroom totals

```
print("Missing before:", housing["total_bedrooms"].isna().sum())
housing["total_bedrooms"] = housing["total_bedrooms"].fillna(housing["total_bedrooms"].median())
print("Missing after:", housing["total_bedrooms"].isna().sum())
```

The first missing count is 207. The median of the observed bedroom totals is calculated, and `fillna` inserts it in the missing positions. The remaining count is zero.

As in the Titanic notebook, this code learns its imputation value from the full table before splitting. A stricter implementation splits first, calculates the training median, and reuses it for validation and test. If ratio features depend on the imputed value, compute those ratios after applying the fitted imputation.

6.7 Why ratios can describe a district differently

```
housing["rooms_per_household"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["population_per_household"] = housing["population"] / housing["households"]

housing.select_dtypes("number").corr()["median_house_value"][
    ["total_rooms", "rooms_per_household", "total_bedrooms", "bedrooms_per_room"]].round(2)
```

Total rooms reflects both district size and housing characteristics. Dividing it by the number of households gives a mean rooms-per-household quantity. Two districts with the same total room count can differ substantially in this ratio if one contains many more households.

The three engineered features are

$$\text{rooms per household} = \frac{\text{total rooms}}{\text{households}},$$

$$\text{bedrooms per room} = \frac{\text{total bedrooms}}{\text{total rooms}}, \quad \text{population per household} = \frac{\text{population}}{\text{households}}.$$

For an invented district with 5,000 rooms and 1,000 households, the first ratio is five rooms per household. If it has 1,000 bedrooms, the bedroom-to-room ratio is 0.2. If its population is 3,000, the population-to-household ratio is three.

These are aggregate ratios. The table does not establish that every household has the resulting average number of rooms or people. Also inspect denominator values: division by zero creates undefined or infinite results, and a very small denominator can create an extreme ratio.

In this dataset, rooms per household has correlation about 0.15 with the target, and bedrooms per room about -0.23 . The latter is stronger in magnitude than the raw bedroom total's correlation of about 0.05. This makes it a candidate feature, but validation must determine how it affects a particular model.

6.8 Encoding ocean categories

```
housing = pd.get_dummies(housing, columns=["ocean_proximity"], dtype=int)
housing.columns.tolist()
```

`get_dummies` creates five indicator columns for the five recorded categories and removes the original text column. The code uses `dtype=int` and retains every category because `drop_first` is not specified.

After adding the three ratios and replacing the categorical field with five indicators, there are 17 columns including the target. The modelling input therefore contains 16 features. `columns.tolist()` displays their names as a Python list.

With an intercept and all five ocean indicators, an ordinary linear model has an exact redundancy: the indicators sum to one. Scikit-learn's least-squares solver can still return a solution, but individual coefficients are not uniquely identifiable under that redundant representation. Interpret the prediction function with care, and use a reference category when you need a cleaner coefficient interpretation.

6.9 The housing split and scaler

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = housing.drop(columns="median_house_value")
y = housing["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler().fit(X_train)      # fit on TRAIN only
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

print("Train:", X_train.shape, "| Test:", X_test.shape)
```

The target is separated from the feature table, and 20 percent is held out using the fixed split seed. Training has 16,512 rows and testing has 4,128 rows. Each input row has 16 features.

The scaler learns from `X_train` and transforms both tables. The original unscaled DataFrames remain available because the scaled results use names ending in `_s`. This is useful: the one-feature model uses income in its original scale, the multifeature linear model uses scaled inputs, and the random forest uses unscaled inputs.

Target values remain in dollars. Scaling the input columns does not automatically scale the target. Consequently, the prediction errors calculated next also have dollar units.

7 Regression metrics: measuring numerical mistakes

Material covered: Slide 22; Notebook 3, metric helper.

7.1 Residuals and why the signs matter

For a row with true target y_i and prediction $\hat{y}_i$, define the residual for these explanations as

$$e_i = \hat{y}_i - y_i.$$

A positive residual means overprediction; a negative residual means underprediction. Simply averaging signed residuals can hide large mistakes because positive and negative values can cancel.

Imagine three true prices of \$100,000, \$200,000, and \$300,000 with predictions of \$120,000, \$180,000, and \$360,000. The residuals are \$20,000, $-\$20,000$, and \$60,000. Their signed mean is \$20,000, but the average magnitude of the mistakes is larger.

7.2 Mean absolute error

Mean absolute error, or MAE, averages the absolute residuals:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|.$$

Absolute value removes each sign. For the three invented prices,

$$\text{MAE} = \frac{20,000 + 20,000 + 60,000}{3} \approx \$33,333.$$

This tells us the average absolute size of a mistake over these examples. It does not mean every prediction is within that distance or that it defines a confidence interval for a new prediction.

MAE uses the target's units. For housing, that is dollars. For temperature prediction, it would be the chosen temperature unit. Comparing MAEs from unrelated tasks without their units and context is usually unhelpful.

7.3 Root mean squared error

Root mean squared error, or RMSE, first squares residuals, averages them, and takes the square root:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}.$$

For the invented example, in thousands of dollars,

$$\text{RMSE} = \sqrt{\frac{20^2 + (-20)^2 + 60^2}{3}} \approx 38.3,$$

so RMSE is about \$38,300. The squaring step gives large errors greater influence. The final square root returns the measure to dollar units.

On the same residuals, RMSE is at least as large as MAE. A large gap between the two can indicate that some unusually large errors contribute strongly. Inspecting the residual distribution helps explain that gap.

7.4 Understanding R-squared

For a non-constant evaluation target, the usual definition is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{\text{eval}})^2}.$$

The numerator is the model's total squared error. The denominator is the total squared error from predicting the evaluation set's own target mean for every row. Thus R^2 measures squared-error reduction relative to that reference.

For the invented three-price example, the mean true price is \$200,000. Using thousands of dollars, the denominator is $100^2 + 0^2 + 100^2 = 20,000$. The model's numerator is $20^2 + (-20)^2 + 60^2 = 4,400$. Therefore $R^2 = 1 - 4,400/20,000 = 0.78$.

A score of one corresponds to perfect agreement. A score of zero matches the reference squared error. A negative value means the model's squared error exceeds that reference. R^2 is dimensionless and is not a percentage of individually correct predictions.

The phrase “explained variation” is often used as shorthand, but it does not establish a causal explanation for why house values differ. Here $R^2 \approx 0.60$ describes roughly a 60-percent reduction in test-set squared error relative to the test-mean reference.

A deployable constant baseline must use a training-derived value. Its mean can differ slightly from the test mean, so its test R^2 may be slightly negative. That is why the supplied baseline can print as -0.00 after rounding.

7.5 The reporting helper, line by line

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def report(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = mean_squared_error(y_true, y_pred) ** 0.5
    r2 = r2_score(y_true, y_pred)
    print(f"{name:32s} MAE ${mae:>9,.0f}    RMSE ${rmse:>9,.0f}    R2 {r2:.2f}")
    return {"model": name, "MAE": round(mae), "RMSE": round(rmse), "R2": round(r2, 3)}
```

The helper receives a model name, true targets, and predicted targets. It calls the metric functions on matching sequences of rows. `mean_squared_error(... ) ** 0.5` takes the square root of MSE to obtain RMSE.

The f-string formats a readable results line. `32s` reserves space for a text label; the numeric formatting inserts thousands separators, controls alignment, and rounds to zero decimal places for the dollar metrics. `.2f` displays R^2 to two decimal places.

The returned dictionary stores the results under named keys. It is separate from the printed sentence. Collecting these dictionaries in a list allows the later comparison table to be built with `pd.DataFrame(results)`. The stored values are rounded for display; the underlying fitted model is unaffected.

Pause and try. If one model has lower MAE but higher RMSE than another, what should you investigate?

Answer: Examine the distribution of its errors. It may improve many ordinary predictions while making a few larger mistakes. The preferred behaviour depends on the application's error costs.

8 Housing models and worked exercises

Material covered: Slides 21–23; Notebook 3, Part 4 and exercises.

8.1 A constant baseline

```
results = []
baseline_pred = np.full(len(y_test), y_train.mean())
results.append(report("Guess the average", y_test, baseline_pred))
```

`results = []` starts an empty collection for comparison rows. `y_train.mean()` calculates the training-set average house value. `np.full(len(y_test), ...)` creates one copy of that same prediction for each of the 4,128 test districts.

The baseline ignores each district's features. It provides a simple point of reference for the added value of learned relationships. Its checked MAE is about \$90,607 and RMSE about \$114,486. Its R^2 is very close to zero, slightly negative before rounding.

The mean is the best constant under squared-error loss on the data used to estimate it. If optimising absolute-error loss, the median is the corresponding constant choice. This notebook deliberately uses the training mean as its baseline.

8.2 Linear regression with median income

```
from sklearn.linear_model import LinearRegression

lin1 = LinearRegression().fit(X_train[["median_income"]], y_train)
pred1 = lin1.predict(X_test[["median_income"]])
results.append(report("Income only", y_test, pred1))

print(f"\nLearned: price = {lin1.coef_[0]:.0f} * income + {lin1.intercept_:,.0f}")
```

Linear regression learns a numerical prediction function. With one input, it has the form

$$\hat{y} = wx + b,$$

where w is a slope and b an intercept. The slope describes the change in predicted output for a one-unit change in the input within this model. The intercept is the prediction at input zero, which may lie outside the useful observed range.

Double brackets around `median_income` preserve a one-column DataFrame. The training call receives that table and the training targets. It fits a line by minimising squared residuals under the ordinary least-squares formulation used here.

Scikit-learn's `LinearRegression` uses a least-squares solving procedure for these dense inputs. The Day 1 gradient-descent loop illustrates one approach to optimisation, but this call does not reproduce that hand-written update loop.

The checked fitted relationship, rounded to whole dollars, is

$$\widehat{\text{house value}} \approx 41,934 \times \text{median income} + 44,460.$$

`lin1.coef_[0]` retrieves the fitted slope, and `lin1.intercept_` retrieves the intercept. Attributes ending in an underscore commonly hold fitted information in scikit-learn.

Using the notebook's income-unit convention, an increase of approximately \$10,000 in district median income corresponds to about \$41,934 higher predicted median house value in this one-feature model. This is an association expressed by the fitted line. It is not a claim about the effect of changing residents' incomes through an intervention.

For an income value of 5, the rounded formula gives $41,934 \times 5 + 44,460 = 254,130$, about \$254,000. The model uses unrounded coefficients internally. On the test set, its MAE is \$62,991, RMSE \$84,209, and $R^2 \approx 0.459$.

8.3 Plot the line over the test examples

```
plt.scatter(X_test["median_income"], y_test, alpha=0.1, s=10, label="real districts")
xs = np.linspace(0, 15, 100).reshape(-1, 1)
plt.plot(xs, lin1.predict(pd.DataFrame(xs, columns=["median_income"])), color="orange",
         linewidth=3, label="learned line")
plt.xlabel("median income (tens of thousands $)")
plt.ylabel("median house value ($)")
plt.legend()
plt.title("Linear regression with one feature")
plt.show()
```

The scatter plot shows true test-set target values against their incomes. `np.linspace(0, 15, 100)` creates 100 evenly spaced incomes for drawing a smooth line. `reshape(-1, 1)` turns them into a one-column input array; `-1` asks NumPy to infer the row count from the total number of values.

The new DataFrame uses the same column name as the fitting input. Predicting for those 100 incomes gives line heights, and `plt.plot` connects them. The actual test observations remain separate dots. Their vertical differences from the line represent residuals.

The broad spread and the capped target band make the line's limitations visible. The line is also drawn down to income zero, even though the observed income field starts above that. Treat this as a display choice rather than evidence about unobserved low-income values.

8.4 Linear regression with all prepared features

```
lin = LinearRegression().fit(X_train_s, y_train)
pred_all = lin.predict(X_test_s)
results.append(report("Linear regression (all)", y_test, pred_all))
```

The multifeature model uses

$$\hat{y} = b + w_1x_1 + w_2x_2 + \cdots + w_px_p,$$

with one coefficient per supplied feature. Here $p = 16$, and the model receives the standardised feature arrays. It combines location, district totals, income, ratios, and category indicators.

The checked MAE is \$50,889, RMSE \$72,669, and $R^2 \approx 0.597$. This improves on the income-only model for these test examples. More features can help when they supply useful information in a form the chosen model can use; there is no general guarantee that each added feature will improve performance.

8.5 Predicted values against recorded values

```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, pred_all, alpha=0.15, s=10)
plt.plot([0, 500000], [0, 500000], color="orange", linewidth=2, label="perfect prediction")
plt.xlabel("real value ($)")
plt.ylabel("predicted value ($)")
plt.legend()
plt.title("Predicted vs real (linear regression)")
plt.show()
```

The horizontal coordinate is the true target, and the vertical coordinate is the prediction. The orange diagonal represents $\hat{y} = y$. A point above it is an overprediction, and a point below it an underprediction. The distance from that equality line helps you inspect the pattern of mistakes.

A vertical stripe at \$500,001 appears because many rows share the capped recorded target. Their predictions can differ because their features differ. A linearly combined prediction can also be negative, even though a negative house value is not meaningful in this context. The model is unconstrained unless the method or application explicitly adds a suitable constraint.

Clipping a prediction at zero is a modelling or postprocessing choice that would need its own evaluation. It does not repair the missing information or assumptions that produced the original prediction.

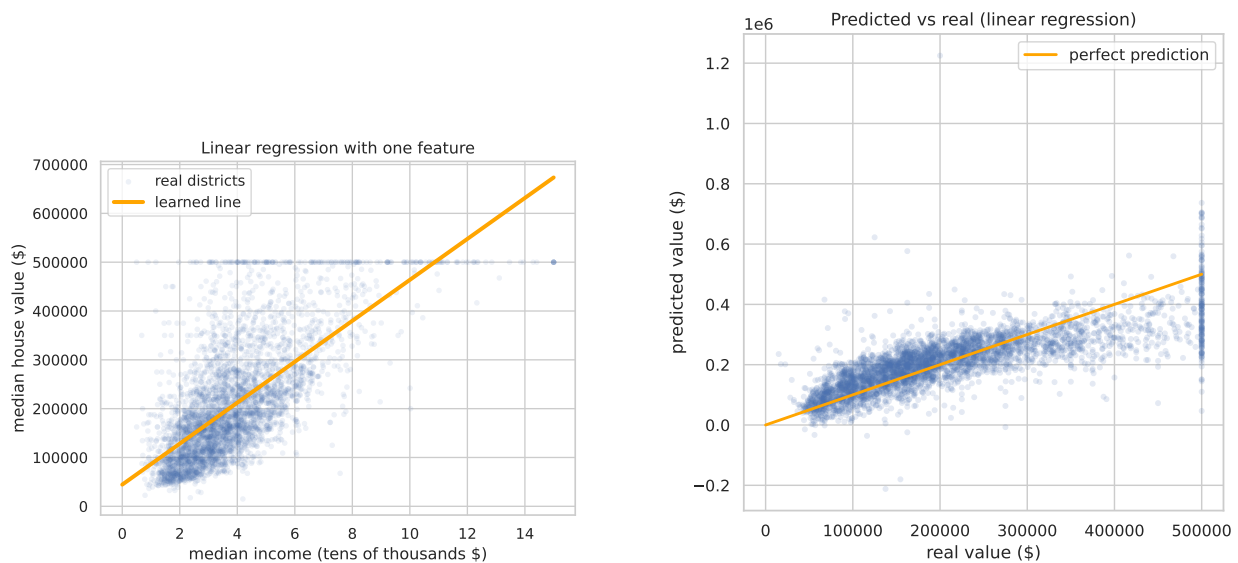


Figure 5: The one-feature fitted line and the multifeature model's predicted-versus-recorded values.

8.6 Reading the linear coefficients

```
coefs = pd.Series(lin.coef_, index=X.columns).sort_values()
coefs.plot(kind="barh", figsize=(7, 6))
plt.title("Linear regression weights (scaled features)")
plt.show()
```

`pd.Series(lin.coef_, index=X.columns)` pairs each fitted coefficient with its feature name. Sorting orders the coefficients, and `kind="barh"` requests horizontal bars.

Because the inputs are standardised, a coefficient gives the model's predicted dollar change for a one-training-standard-deviation change in that input, holding the other represented inputs fixed. The sign describes the direction of that conditional model association.

Coefficient magnitude is not a universal measure of feature importance. Correlated inputs, exact dependencies, rare category indicators, outliers, and the model specification affect it. The housing table includes district totals, ratios derived from those totals, and all ocean indicators with an intercept. Some hypothetical changes that hold every other feature fixed may not correspond to a consistent real district.

Use the plot to investigate what the fitted linear function is doing. Do not read it as a ranked list of independent causes of property value.

8.7 A random-forest preview

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1).fit(X_train, y_train)
results.append(report("Random forest (Day 3 preview)", y_test, rf.predict(X_test)))
```

A random forest combines many decision trees. Each tree is trained using randomisation, and the forest averages their numerical predictions for regression. Splits allow different rules in different regions of feature space, so the model can represent nonlinear relationships and interactions.

`n_estimators=100` requests 100 trees. `random_state=42` controls random choices for reproducibility. `n_jobs=-1` allows use of available CPU workers for supported parallel operations. Actual runtime depends on the hardware and environment, so the cell may take different amounts of time on different computers.

The notebook uses unscaled training inputs for the forest. Standardisation is generally unnecessary for conventional tree splits. The checked forest MAE is \$32,338, RMSE \$50,383, and $R^2 \approx 0.806$ on the same held-out rows.

The result motivates the deeper study of tree ensembles on Day 3. It does not establish that this model will be best on another dataset or on geographically separate districts.

8.8 Read the complete comparison

```
pd.DataFrame(results).set_index("model")
```

Original notebook experiment	MAE (\$)	RMSE (\$)	R^2
Training-mean baseline	90,607	114,486	≈ 0.000
Income only	62,991	84,209	0.459
Linear regression, all inputs	50,889	72,669	0.597
Random forest preview	32,338	50,383	0.806

All rows are evaluated on the same 4,128 test districts. `set_index("model")` uses the model names as row labels. Lower MAE and RMSE represent smaller errors under those measures, while larger R^2 represents greater squared-error reduction relative to its reference.

These results reproduce the original full-data imputation workflow. They should be labelled accordingly. Moving all learned preparation inside the training boundary creates a revised experiment whose scores should be calculated and reported separately.

8.9 Housing Exercise 1: income plus latitude

```
# Exercise 1
two = ["median_income", "latitude"]
lin2 = LinearRegression().fit(X_train[two], y_train)
report("Income + latitude", y_test, lin2.predict(X_test[two]));
```

The list `two` selects the two input columns. The resulting model estimates a coefficient for income and another for latitude. It is fitted and tested on the corresponding subsets without standardisation in this exercise.

The checked MAE is \$62,519, RMSE \$83,632, and $R^2 \approx 0.466$. This is a small improvement over income alone, whose R^2 is about 0.459. A single linear latitude term offers only a restricted representation of location; it cannot capture every regional pattern seen in the map.

The final semicolon can suppress automatic display of the returned dictionary in a notebook. The helper's explicit print statement still displays its metrics.

8.10 Housing Exercise 2: remove the three ratios

```
# Exercise 2
ratio_cols = ["rooms_per_household", "bedrooms_per_room", "population_per_household"]
Xtr_no = X_train.drop(columns=ratio_cols); Xte_no = X_test.drop(columns=ratio_cols)
sc_no = StandardScaler().fit(Xtr_no)
lin_no = LinearRegression().fit(sc_no.transform(Xtr_no), y_train)
report("Linear, no engineered features", y_test, lin_no.predict(sc_no.transform(Xte_no)))
report("Linear, with engineered features", y_test, pred_all);
# Surprise! For LINEAR regression the ratio features slightly HURT (R2 0.63 without vs 0.60 with
# ).
# A few districts have extreme ratios (e.g. over 100 rooms per household), and a straight line
# is pulled around by such outliers. Lesson: a feature that looks useful in EDA must still be
# tested on the model - and different models benefit from different features (trees handle them
# well).
print("Max rooms_per_household:", round(X_train["rooms_per_household"].max(), 1))
```

The code drops the ratio columns from both training and test tables. A new scaler is fitted to the reduced training table, and a new linear model is fitted to its transformed values. A different feature set requires a corresponding transformation with the right number and order of columns.

The observed comparison is:

Linear model	MAE (\$)	RMSE (\$)	R^2
Without engineered ratios	50,671	70,061	approximately 0.63
With engineered ratios	50,889	72,669	approximately 0.60

The ratios slightly worsen the measured result, especially RMSE. The maximum training rooms-per-household value is about 141.9, which shows that some ratios are extreme. Large input values can exert substantial influence on a least-squares fit.

Extreme ratios are a plausible contributor to this behaviour, as the speaker notes suggest. This comparison alone does not isolate the exact cause: correlated features, model form, and sampling also affect the result. Investigate the unusual rows and test any proposed remedy within validation data.

A feature's interesting correlation does not guarantee an improvement for every estimator. Keep the hypothesis and the measured result separate. The full-data EDA suggests possible features; model evaluation determines whether the chosen representation helped under the experiment.

8.11 Housing Exercise 3: change a district's income input

```
# Exercise 3
new_district = X_test.iloc[[0]].copy()
print("Original prediction:", f"${rf.predict(new_district)[0]:,.0f}")
new_district["median_income"] = 8.0
print("With income = 8.0: ", f"${rf.predict(new_district)[0]:,.0f}")
```

`X_test.iloc[[0]]` selects the first test row by position while keeping it as a one-row DataFrame. `copy()` allows the new row to be edited without changing the original test table.

The forest's original prediction for that row is about \$48,444. After replacing its median-income input with 8.0 and leaving other inputs as written, the prediction becomes about \$305,441. The forest sees different feature values and follows different learned branches.

This is a model sensitivity exercise. It does not estimate the causal effect of raising income, and the modified combination of characteristics may not describe an actual district. There is also no known target for the edited row, so this exercise cannot measure its prediction error.

9 Titanic classification: from a score to a decision

Material covered: Slides 24–26; Notebook 2, Part 7.

9.1 Return to the prepared passenger data

The Titanic inputs have already been selected, imputed, encoded, split, and scaled in Notebook 2. The target is `survived`, with zero for a non-survivor and one for a survivor. The original split contains 712 training passengers and 179 test passengers.

A classifier assigns a category to an input. Some classifiers also provide a probability estimate or score that can be used to make the decision. Keep the input table, the fitted preprocessing, the model, and the interpretation of its output together.

9.2 The majority-class baseline

```
baseline_acc = (y_test == 0).mean()
print(f"Baseline (always 'died') accuracy: {baseline_acc:.1%}")
```

The expression `y_test == 0` produces a true value for each test passenger who did not survive. Taking its mean gives the fraction of test passengers in class zero. The checked value is $110/179 \approx 0.6145$, displayed as 61.5 percent.

This is the accuracy of predicting “did not survive” for every test passenger. It requires no individual feature information. The result provides a reference for judging how much the fitted classifiers add.

For a formal baseline, choose the constant class from training data and then evaluate it on the test data. Zero is the majority class in this training split, so the classroom calculation corresponds to that majority-class strategy. It should not be selected by looking for the most common class in the test set.

Accuracy alone describes only how often a label matches. The constant-zero baseline identifies none of the 69 survivors, so its recall for survivors is zero. Whether a baseline must be exceeded on accuracy or on another measure depends on the actual objective.

9.3 What logistic regression computes

Logistic regression is a classification method that converts a linear score into a probability estimate. For prepared features $x_1, \dots, x_p$, it first calculates

$$z = b + w_1x_1 + w_2x_2 + \dots + w_px_p.$$

The **sigmoid function** maps this score to a value between zero and one:

$$p = \frac{1}{1 + e^{-z}}.$$

Here e is the mathematical constant approximately equal to 2.718. As z grows, the output approaches one; as z becomes strongly negative, it approaches zero. At $z = 0$, the output is 0.5.

For example, $z = 2$ produces a probability estimate of about 0.881, and $z = -2$ produces about 0.119. These are examples of the transformation, not the exact scores of the two fictional passengers used later.

In this model, p estimates the probability of class one under the fitted relationship and dataset. A standard binary decision uses the 0.5 boundary. Moving the boundary changes which probability estimates produce a positive label without changing the fitted coefficients.

The name logistic regression relates to modelling the **log-odds** linearly. Odds are $p/(1 - p)$, and their natural logarithm is

$$\log\left(\frac{p}{1 - p}\right) = z.$$

At $p = 0.8$, the odds are $0.8/0.2 = 4$. You can follow the notebook by understanding the score and sigmoid first; log-odds become useful when interpreting the fitted coefficients more deeply.

9.4 Fit the classifier and make predictions

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay,
    classification_report

logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train_s, y_train)
pred = logreg.predict(X_test_s)

print(f"Accuracy: {accuracy_score(y_test, pred):.1%}")
```

The import brings in the classifier and several evaluation utilities. The classifier is created with `max_iter=1000`, permitting up to 1,000 solver iterations. The solver may stop earlier after meeting

its convergence criterion. This setting is a limit on optimisation work, not a request to train 1,000 independent models.

`fit(X_train_s, y_train)` estimates the coefficients using the scaled training inputs and their labels. The default estimator also uses regularisation, which constrains the fitted coefficients according to its settings. This is one reason input scale matters for the fitted result.

`predict(X_test_s)` produces one label per test row. `accuracy_score(y_test, pred)` compares these labels with the true outcomes in the same order. The checked result is 80.4 percent, corresponding to 144 correct predictions out of 179.

This improves on the 61.5-percent baseline by about 19 percentage points. A percentage-point difference subtracts two percentages directly. It is distinct from a relative percentage increase calculated by dividing that difference by the starting percentage.

9.5 How KNN and a decision tree use these inputs

KNN compares a new passenger's feature vector with the training vectors. With $k = 5$, the classifier considers five nearby examples and uses their labels to vote. The scaled feature space determines the distances, so the representation strongly affects which passengers count as similar.

A decision tree chooses feature thresholds that separate training labels into more homogeneous groups. Each input follows branches until reaching a leaf, whose class information determines the prediction. The slide's flowchart is a simplified illustration; the fitted tree may contain more detailed rules.

The depth-four tree can use at most four successive splits on a path from the root. A depth restriction limits complexity, although it does not specify the exact number of leaves that will be created. KNN's neighbour count and the tree's depth are examples of hyperparameters chosen outside the fitting of individual coefficients or splits.

10 The confusion matrix and classification metrics

Material covered: Slides 27–29; Notebook 2, Parts 7.3–7.5.

10.1 Start with the actual counts

```
cm = confusion_matrix(y_test, pred)
ConfusionMatrixDisplay(cm, display_labels=["died", "survived"]).plot(cmap="Blues")
plt.title("Logistic regression on 179 test passengers")
plt.show()
```

The function `confusion_matrix` counts combinations of true and predicted labels. In this binary example, rows represent true outcomes and columns represent predictions, both ordered as zero then one. The display labels replace those codes with “died” and “survived”.

	Predicted died	Predicted survived	Row total
Actually died	97	13	110
Actually survived	22	47	69
Column total	119	60	179

The diagonal entries are correct predictions: 97 non-survivors and 47 survivors. The off-diagonal entries are mistakes: 13 non-survivors labelled as survivors and 22 survivors labelled as non-survivors.

We designate survival as the **positive class**. “Positive” identifies the category whose detection we are discussing; it does not mean that the prediction is correct. With this choice:

True positive, TP = 47

Predicted survival for someone who survived.

True negative, TN = 97

Predicted non-survival for someone who did not survive.

False positive, FP = 13

Predicted survival for someone who did not survive.

False negative, FN = 22

Predicted non-survival for someone who survived.

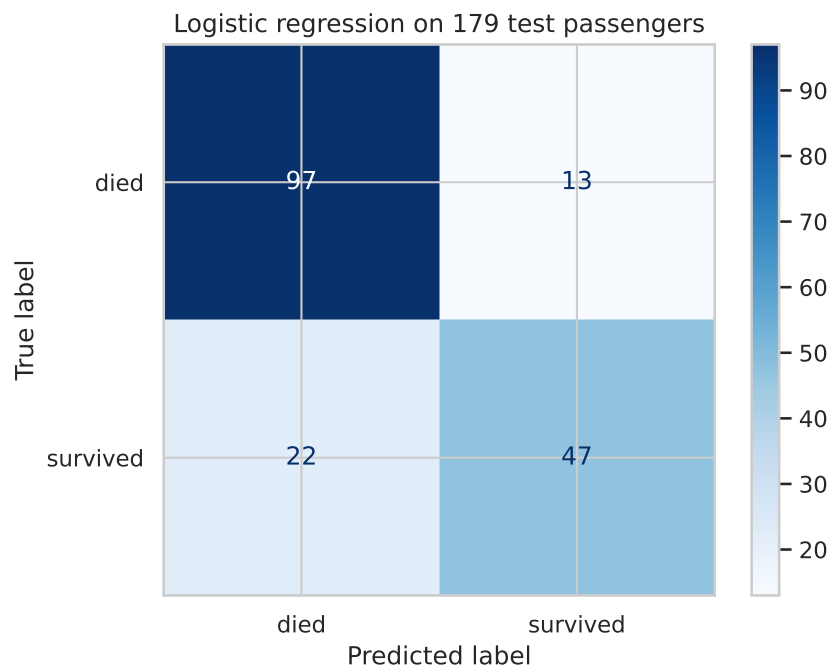


Figure 6: The original logistic-regression confusion matrix on 179 test passengers.

Before applying any formula, identify the positive class and confirm the matrix orientation. Other plotting conventions may place predictions on rows, so reading the labels prevents swapping false positives and false negatives.

10.2 Accuracy: the overall fraction correct

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{47 + 97}{179} \approx 0.8045.$$

This is about 80.4 percent. It treats every correctly classified passenger equally, regardless of class. It is useful as an overall summary, but it does not describe how errors are divided between classes.

A model can achieve a high accuracy when one class dominates even if it rarely detects the other class. Always interpret accuracy with the class counts and the consequences of the two kinds of mistake.

10.3 Precision: reliability among positive predictions

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{47}{47 + 13} = \frac{47}{60} \approx 0.7833.$$

The model predicted survival for 60 passengers. Of these, 47 actually survived, so its precision for survival is about 78.3 percent. Precision starts from the set of examples predicted positive.

A false positive adds to the denominator without adding to the numerator, lowering precision. In an alerting application, higher precision means a larger fraction of triggered alerts correspond to the selected event, under the evaluated conditions.

Precision is undefined if no positive predictions are made because its denominator is zero. Software can report a chosen convention, often zero with a warning depending on settings. The baseline table's dash for survivor precision reflects that problem: the constant-zero classifier never predicts survival.

10.4 Recall: coverage of the actual positive cases

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{47}{47 + 22} = \frac{47}{69} \approx 0.6812.$$

There are 69 actual survivors in the test set, and the model recognises 47 of them. Its recall for survival is about 68.1 percent. Recall starts from the examples that are truly positive.

A false negative is an actual positive that the model misses. Increasing recall means finding a larger share of the positive examples, although the change can bring additional false positives. The appropriate balance depends on the use of the predictions.

To keep the denominators straight, read the questions aloud: precision asks how many predicted survivors were survivors; recall asks how many actual survivors were found. The numerator is the same 47, but the two reference groups differ.

10.5 F1: combining precision and recall

The F1 score is their harmonic mean:

$$F_1 = \frac{2PR}{P + R}.$$

For this confusion matrix it can also be written directly as

$$F_1 = \frac{2TP}{2TP + FP + FN} = \frac{94}{94 + 13 + 22} = \frac{94}{129} \approx 0.7287.$$

Thus the survivor F1 is about 72.9 percent. F1 is low when either precision or recall is low. It does not include true negatives in this binary positive-class formula, so it is not a complete description of all performance.

A metric selects what to emphasise. F1 is useful when both positive-class precision and recall matter, but a real decision may require explicit error costs, coverage requirements, or other measures. There is no single metric that automatically captures every application's needs.

10.6 Reading the classification report

```
print(classification_report(y_test, pred, target_names=["died", "survived"]))
```

The report gives precision, recall, F1, and support for each class. **Support** is the number of true examples in that class: 110 for died and 69 for survived.

For the died row, the calculation temporarily treats died as the class of interest. Its precision is $97/(97 + 22) \approx 81.5$ percent, and its recall is $97/(97 + 13) \approx 88.2$ percent. These differ from the survivor metrics because the class role has changed.

Macro average calculates a metric for each class and gives the classes equal weight. **Weighted average** weights each class's metric by its support. For example, the weighted recall combines the class recalls using 110 and 69 as weights; in this ordinary single-label classification setting it equals accuracy.

The report prints rounded decimals. Small apparent disagreements with hand calculations can be due to display precision. Reconstructing a metric from the integer confusion counts avoids unnecessary rounding error.

10.7 Interpreting the logistic coefficients

```
weights = pd.Series(logreg.coef_[0], index=X.columns).sort_values()
weights.plot(kind="barh")
plt.title("Logistic regression weights")
plt.show()
```

`logreg.coef_` has one row of coefficients for this binary classifier. Selecting `[0]` retrieves that row. Pairing it with `X.columns` produces a named Series, sorting orders the values, and `barh` draws horizontal bars.

A positive coefficient raises the linear log-odds score when that feature increases and other represented inputs are held fixed. A negative coefficient lowers it. Because the sigmoid increases with the score, the same directions apply to the model's survival probability estimate.

In the checked model, the scaled sex coefficient is about $+1.234$, ticket class about -0.892 , and age about -0.487 . Since female was encoded as one and male as zero, the positive sex coefficient agrees with the direction of the grouped rates. A larger ticket-class code and a greater age have negative conditional associations in this fitted model.

The coefficients act on standardised features. A coefficient corresponds to a one-unit change in that standardised coordinate, not necessarily to one original year or a complete binary switch. For a binary feature, changing its original value from zero to one spans a number of standardised units determined by its training standard deviation.

Scaling helps place features on comparable numerical spreads, but coefficient size alone does not prove importance or causation. The family-size feature is derived from two other retained counts, and related predictors can share influence. Regularisation and the chosen input representation also affect the fitted values.

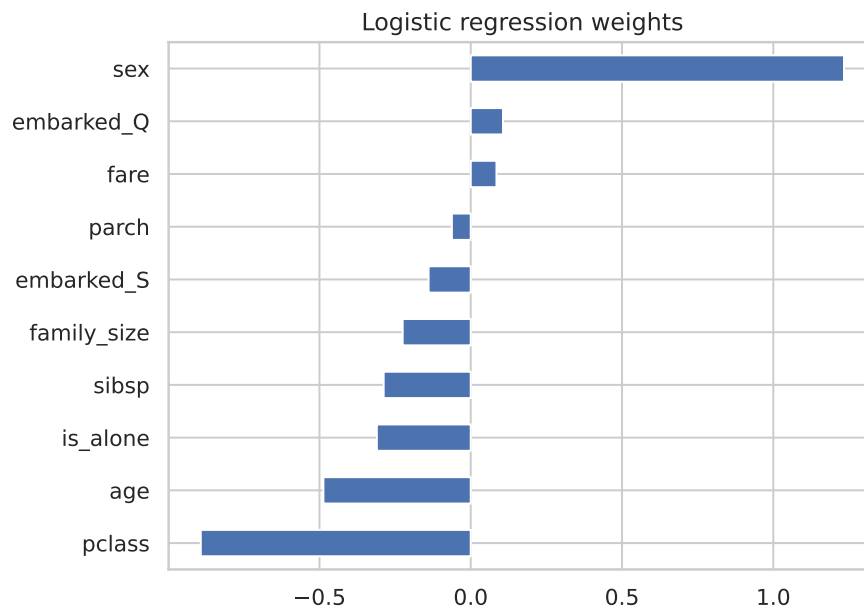


Figure 7: Logistic-regression coefficients attached to the scaled input features.

10.8 The model-comparison loop

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score, recall_score, f1_score

models = {
    "Logistic regression": LogisticRegression(max_iter=1000),
    "KNN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Decision tree (depth 4)": DecisionTreeClassifier(max_depth=4, random_state=42),
}

rows = []
for name, model in models.items():
    model.fit(X_train_s, y_train)
    p = model.predict(X_test_s)
    rows.append({
        "model": name,
        "accuracy": accuracy_score(y_test, p),
        "precision": precision_score(y_test, p),
        "recall": recall_score(y_test, p),
        "f1": f1_score(y_test, p),
    })

results = pd.DataFrame(rows).set_index("model").round(3)
results
```

The dictionary `models` associates each display name with an estimator. `models.items()` supplies pairs of names and objects to the loop. Each estimator is fitted on the same training arrays, then predicts the same test rows.

For each model, a dictionary records four evaluation metrics. Appending these dictionaries to `rows` builds a list suitable for creating a results table. `set_index("model")` makes the names the row labels, and `round(3)` rounds the table for display.

The same scaled arrays are used for all three classifiers for convenience. The tree does not require

standardisation, but monotonic scaling preserves the ordering used for its feature thresholds, subject to numerical details.

Model	Accuracy	Precision	Recall	F1
Always predict died	61.5%	undefined	0.0%	0.0%
Logistic regression	80.4%	78.3%	68.1%	72.9%
KNN, $k = 5$	81.6%	79.0%	71.0%	74.8%
Tree, maximum depth 4	78.8%	84.4%	55.1%	66.7%

Precision, recall, and F1 in this table refer to the survivor class. For the constant-negative baseline, the count-based F1 formula gives zero because there are actual positives but no true positives. This is why its F1 can be given as zero even though its precision has an undefined denominator.

10.9 What would justify choosing one model?

Among these three fitted models on this split, KNN has the highest recall, accuracy, and F1, while the depth-four tree has the highest precision. If the immediate exercise prioritises finding actual survivors, recall points toward KNN. If the concern is the reliability of positive predictions, the tree's precision deserves inspection together with how many survivors it misses.

The dataset contains only 179 test passengers. One additional correct prediction changes accuracy by about $100/179 = 0.56$ percentage points. The KNN–logistic accuracy difference corresponds to two passengers. That is modest evidence for a general ranking.

Use validation or cross-validation to compare candidate models and settings, and reserve independent test data for final assessment. The notebook's side-by-side test comparisons are educational demonstrations. Once their results guide choices, those examples have influenced development.

11 Overfitting, new passengers, and threshold experiments

Material covered: Slide 30; Notebook 2, Parts 7.6–8 and exercises.

11.1 Compare training and test performance

```
for depth in [None, 4]:
    tree = DecisionTreeClassifier(max_depth=depth, random_state=42).fit(X_train_s, y_train)
    label = "no limit" if depth is None else f"max depth {depth}"
    print(f"Tree ({label:12s}) train: {tree.score(X_train_s, y_train):.1%}    test: {tree.score(
        X_test_s, y_test):.1%}")
```

The loop fits trees with two depth settings. `None` permits growth subject to the estimator's other stopping conditions; a value of four restricts maximum depth. The conditional expression assigns a readable name to each case.

The checked results are:

Tree	Training accuracy	Test accuracy
No maximum depth	98.2%	79.9%
Maximum depth 4	83.8%	78.8%

The unrestricted tree fits the training examples very closely, with a much lower score on held-out examples. This is a warning sign consistent with overfitting: learning patterns too specific to the training set, including noise or quirks, that transfer poorly.

The shallower tree has a smaller training–test gap. Its test accuracy on this split is also slightly lower than the unrestricted tree’s. The smaller gap alone therefore does not establish that it is the better predictor. Evaluate the relevant objective on appropriate validation data when choosing complexity.

A gap can also reflect differences between datasets, sampling variation, or evaluation problems. Use it as evidence to investigate, not as a complete diagnosis in isolation. Day 3 introduces further methods for controlling complexity and estimating performance.

11.2 Why even an unrestricted model may miss training examples

The no-depth-limit tree achieves 98.2 percent rather than 100 percent training accuracy. Rows can share the same prepared feature values while having different labels. A deterministic predictor receiving identical inputs cannot return two different labels for those otherwise indistinguishable rows.

Similarly, KNN with one neighbour need not obtain a perfect training score when duplicate or tied feature vectors with conflicting targets are present. Preprocessing and the information available in the features affect what can be distinguished.

11.3 Prepare two fictional passengers

```
new_passengers = pd.DataFrame([
    # pclass, sex(1=female), age, sibsp, parch, fare, family_size, is_alone, embarked_Q,
    embarked_S
    {"pclass": 1, "sex": 1, "age": 17, "sibsp": 1, "parch": 1, "fare": 100.0,
     "family_size": 3, "is_alone": 0, "embarked_Q": 0, "embarked_S": 1},
    {"pclass": 3, "sex": 0, "age": 20, "sibsp": 0, "parch": 0, "fare": 7.9,
     "family_size": 1, "is_alone": 1, "embarked_Q": 0, "embarked_S": 1},
], index=["1st-class girl, 17", "3rd-class man, 20"])[X.columns]

probs = logreg.predict_proba(scaler.transform(new_passengers))[:, 1]
for name, p in zip(new_passengers.index, probs):
    print(f"{name}: {p:.0%} chance of survival")
```

The first row describes a 17-year-old female passenger in first class with one sibling or spouse, one parent or child, fare 100, and embarkation code S. Her family size is three and her alone indicator is zero. The second describes a 20-year-old male passenger in third class with no counted relatives and fare 7.9, also embarking at S.

Both rows explicitly provide the ten features used in training. Selecting `[X.columns]` places them in the training column order. Even when two tables contain the same named fields, careless reordering can cause prediction errors or feature-name checks to fail. Derived features must also be consistent with their original inputs.

`scaler.transform(new_passengers)` applies the training-fitted numerical transformation. The model then calls `predict_proba`, which returns a probability estimate for each class for each row. For this fitted binary classifier, `logreg.classes_` is ordered as `[0, 1]`, so selecting `[:, 1]` takes the survival probabilities for every row.

The `zip` loop pairs each passenger’s descriptive row index with the corresponding estimated

probability. The format `.0%` displays a whole-number percentage. The checked estimates round to 95 percent and 11 percent.

These are model estimates for fictional feature combinations. Their interpretation depends on the dataset, representation, and calibration of the model. They are not known outcomes or guarantees. A probability near 95 percent does not establish that 95 of 100 similarly scored cases would survive without checking calibration on suitable data.

11.4 Exercise 1: compare four neighbour counts

```
# Exercise 1
for k in [1, 5, 15, 30]:
    knn = KNeighborsClassifier(n_neighbors=k).fit(X_train_s, y_train)
    print(f"k={k:2d}  train {knn.score(X_train_s, y_train):.1%}  test {knn.score(X_test_s,
    y_test):.1%}")
# k=1 memorises the training set (look at the train score) - another example of overfitting.
```

k	Training accuracy	Test accuracy
1	98.2%	78.2%
5	85.0%	81.6%
15	82.9%	83.2%
30	81.5%	82.1%

Among these settings on this split, $k = 15$ has the highest test accuracy. With one neighbour, predictions are very local and fit the training examples closely. Larger neighbourhoods average over more training labels, which can reduce sensitivity to individual records but can also blur distinctions.

The $k = 15$ test score is slightly higher than its training score. This can happen because the two evaluated collections differ. A small reverse gap is not itself a problem or proof of unusually strong generalisation.

Use these outputs to understand the effect of k . A formal selection procedure would choose it using validation data or cross-validation. The best value in this table is not guaranteed to be best on future passengers or another dataset.

11.5 Exercise 2: add fare per person

```
# Exercise 2
X2 = X.copy()
X2["fare_per_person"] = X2["fare"] / X2["family_size"]
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y, test_size=0.2, random_state=42,
    stratify=y)
sc2 = StandardScaler().fit(X2_train)
lr2 = LogisticRegression(max_iter=1000).fit(sc2.transform(X2_train), y2_train)
print(f"With fare_per_person: {lr2.score(sc2.transform(X2_test), y2_test):.1%}  (before: {
    accuracy_score(y_test, pred):.1%})")
# Not every new feature helps! That's normal - feature engineering is trial and error.
```

`X.copy()` creates a separate feature table, and the new ratio divides fare by family size. The same split settings are applied to corresponding rows, making this comparison use the same passenger

partition. A new scaler learns the statistics for the expanded feature table, and a new logistic classifier is fitted.

The checked accuracy remains 80.4 percent, matching the original classifier when rounded to one decimal place. An added feature can change probability estimates or individual decisions without changing the total number of correct predictions. The unchanged accuracy is therefore narrower evidence than “nothing changed”.

The ratio is a proposed representation of information already partly present in fare and family size. Its usefulness depends on the model, data, and evaluation criterion. Check additional metrics or prediction changes if those matter to the intended use, while keeping development decisions inside the validation process.

11.6 Exercise 3: raise the decision threshold

```
# Exercise 3
probs_test = logreg.predict_proba(X_test_s)[: , 1]
for threshold in [0.5, 0.7]:
    p = (probs_test > threshold).astype(int)
    print(f"threshold {threshold}: precision {precision_score(y_test, p):.1%}, recall {
    recall_score(y_test, p):.1%}")
# A higher threshold = more careful: higher precision, lower recall.
```

The model is fitted once. `predict_proba(X_test_s)[: , 1]` obtains its survival probabilities for the test rows. Comparing them with a threshold produces Boolean decisions, and `astype(int)` converts those to zero-or-one labels.

With the notebook’s strict greater-than comparison, a value exactly equal to the threshold is assigned zero. Ties at a boundary should be defined explicitly in applications. The results shown here are unaffected by any relevant exact boundary tie.

The checked comparison is:

Threshold	TP	FP	Precision	Recall
0.5	47	13	78.3%	68.1%
0.7	37	5	88.1%	53.6%

Raising the threshold removes some positive decisions. Here it reduces false positives from 13 to 5 and true positives from 47 to 37. Ten additional survivors are missed, so recall falls. Precision rises because the remaining positive predictions contain a larger proportion of actual survivors.

On a fixed set of probability scores, increasing the threshold cannot add predicted positives, so recall cannot increase. Precision often improves but is not guaranteed to increase at every threshold on a finite dataset; it depends on the labels of the cases removed. The particular increase in this exercise is an observed result.

Choosing a threshold is a development decision. Select it using validation data and the intended error costs, then evaluate the fixed choice independently. Do not tune it repeatedly on the final test set and describe that same set as untouched.

Pause and try. What changes when the threshold moves from 0.5 to 0.7: the fitted coefficients, the predicted probabilities, or the final class decisions?

Answer: The class decisions change. The fitted coefficients and the already computed probabilities remain the same in this exercise.

12 A complete training-only preparation example

Material covered: Companion clarification for slide 19 and the preparation stages in Notebooks 2 and 3.

12.1 Why this companion example is included

The supplied notebooks teach a correct principle: fit learned preprocessing on the training data. Their scalers follow that rule, but their earlier missing-value calculations use the complete tables. This example implements the principle consistently for Titanic without changing the supplied files.

A **pipeline** stores a sequence of transformations and a final estimator. Calling its **fit** method fits those steps in sequence using the supplied training examples. Calling its **predict** method applies the fitted transformations before invoking the fitted model. This makes it easier to reuse the exact preparation at evaluation and prediction time.

A **ColumnTransformer** applies different procedures to different named columns. Here numerical inputs receive median imputation and standardisation. The port field receives most-frequent imputation, one-hot encoding, and standardisation of the resulting indicators. Scaling the indicators maintains the broad representation used in the original notebook.

12.2 Load, construct fixed row features, and split

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

raw = sns.load_dataset("titanic")
columns = ["pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]
X = raw[columns].copy()
y = raw["survived"]
# Fixed row-wise rules use no statistics learned from other rows.
X["sex"] = X["sex"].map({"male": 0, "female": 1})
X["family_size"] = X["sibsp"] + X["parch"] + 1
X["is_alone"] = (X["family_size"] == 1).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
numeric_columns = ["pclass", "sex", "age", "sibsp", "parch",
                  "fare", "family_size", "is_alone"]
```

The permissible raw inputs are explicitly selected, so the outcome-revealing **alive** field cannot enter the model. The sex mapping and family calculations are fixed row-wise rules. They use no statistic calculated across passengers.

The split occurs while age and port still contain their original missing entries. Therefore the fitted preparation stages that follow see only training rows when calculating medians, modes, categories, and scales.

12.3 Describe the preparation steps

```
numeric_steps = Pipeline([
    ("fill", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
category_steps = Pipeline([
    ("fill", SimpleImputer(strategy="most_frequent")),
    ("encode", OneHotEncoder(drop="first", handle_unknown="ignore",
                             sparse_output=False)),
    ("scale", StandardScaler()),
])
prepare = ColumnTransformer([
    ("numbers", numeric_steps, numeric_columns),
    ("ports", category_steps, ["embarked"]),
])
```

Each pair in a pipeline contains a step name and the object performing that step. The names help identify the components when inspecting or adjusting the pipeline. `SimpleImputer(strategy="median")` learns one training median for each selected numerical column, although only age needs filling in this selected dataset.

The categorical imputer learns the most frequent training port. `OneHotEncoder` learns the training category vocabulary. `drop="first"` requests a reference category, and `sparse_output=False` requests a dense array compatible with the scaler used in this small example.

`handle_unknown="ignore"` lets an unseen port category produce an all-zero indicator pattern rather than raising an error. With a dropped reference category, that pattern is also the representation of the reference. This is a deliberate fallback to understand, not a claim that an unseen category is actually the reference port. A deployed application should decide how to report or handle unexpected categories.

The `ColumnTransformer` joins the prepared numerical and port outputs into the feature array expected by the classifier. Column selectors are stored with the preparation, reducing the risk of accidentally applying a transformation to the wrong fields.

12.4 Fit the complete sequence and evaluate it

```
classifier = Pipeline([
    ("prepare", prepare),
    ("model", LogisticRegression(max_iter=1000)),
])
classifier.fit(X_train, y_train)
predictions = classifier.predict(X_test)
print("Training-only preparation accuracy:",
      round(accuracy_score(y_test, predictions), 4))
print(confusion_matrix(y_test, predictions))
```

The final pipeline combines preparation and logistic regression. Only `X_train` and `y_train` are supplied to `fit`. The later prediction call receives the raw-format test table, and the pipeline transforms it using the already fitted components.

The companion script is included in the source bundle as `Day2_Training_Only_Titanic.py`. Its checked accuracy is 0.8045, with the same confusion counts as the original experiment in this environment. Run it separately from the original notebook if you want to compare workflows. Its

purpose is to make the training boundary explicit. A corrected workflow can produce the same rounded score as a shortcut workflow on a particular split; correctness of the evaluation design is still a separate issue.

The test set remains independent only while it is excluded from choices. A pipeline prevents certain preprocessing mistakes, but it does not prevent someone from repeatedly selecting features or thresholds using test results. Validation discipline is still required.

12.5 Apply the same principle to housing

For housing, split the raw permissible predictors and target first. Fit the bedroom imputer on training rows. Apply that fitted imputer to both subsets, create each row's ratios with denominator checks, learn the ocean-category encoder on training data, and fit any required scaler on training data. Then fit and evaluate the estimator.

When using cross-validation, each training fold needs its own fitted preparation. A pipeline can arrange that automatically when passed to the validation procedure. Calculating one median or scaler from all rows before the folds would allow information from validation folds into preparation.

13 The complete recipe and what each stage means

Material covered: Slide 31; connections across all three notebooks.

13.1 Read the short slide recipe with its assumptions

The slide gives a compact split, scale, fit, evaluate sequence:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression().fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

This assumes that **X** already contains appropriate numerical features, **y** contains the correct paired labels, and any earlier learned preprocessing respected the same training boundary. The full Titanic notebook adds stratification and a larger solver-iteration limit.

Assigning scaled arrays back to **X_train** and **X_test** replaces their previous values. The full notebooks retain both versions by using separate names ending in **_s**. Either approach can work if the variable meanings remain clear.

For a regression task, the structure still includes fitting and prediction, but evaluation uses numerical-error metrics. A classifier's **score** commonly returns accuracy, while **LinearRegression.score** returns R^2 . The method name alone does not tell you that every estimator uses the same metric.

13.2 A practical sequence for an independent project

1. State the target and what inputs will actually be available when making a prediction.
2. Inspect the source, units, schema, missingness, and relationships between records.
3. Reserve evaluation data in a way that matches the intended deployment, using time, group, or geographical boundaries when appropriate.
4. Explore and develop features using the training and validation portions.
5. Fit learned preparation within each training portion and apply it consistently elsewhere.
6. Fit a baseline and candidate models, then use validation to make choices.
7. Evaluate the selected fixed workflow on the reserved test set, reporting sample counts and relevant metrics.
8. When deploying, store the preparation and model together and monitor their behaviour under real inputs.

The course notebooks provide small, inspectable examples of these operations. Their charts, values, and comparisons help you understand what happens at each stage. An independent project's evaluation needs to match its own data collection and intended use.

14 Common problems and how to understand them

Material covered: The speaker-note troubleshooting guide and the stateful operations in all three notebooks.

14.1 A missing file or failed data download

A CSV loader needs a valid path or reachable URL. If `sns.load_dataset` or `pd.read_csv(url)` fails with a connection or HTTP error, first check the network and source address. Later cells cannot recover a table that the loading cell never created.

A manually uploaded CSV can be loaded from its local filename. Verify that it is the intended dataset and that its column names and values agree with the notebook. A different dataset named Titanic or housing can have a different schema, split, or preprocessing history.

14.2 An encoding cell fails after it previously worked

The sex mapping expects the strings male and female. Running that mapping again after it has already converted the column to zeroes and ones supplies values absent from the mapping dictionary. Those unmapped values become missing.

Similarly, `get_dummies` removes the original categorical column and creates new indicator columns. Running the same encoding statement again can raise `KeyError` because `embarked` or `ocean_proximity` has already been removed. Reload the raw table and rerun the preparation in order. Repeated execution changes state.

14.3 A new input has the wrong columns

A model expects the same representation used during fitting. Check the number, names, order, units, and meanings of the fields. If the fitted model used ten inputs, adding a new feature only to the prediction row gives an incompatible input.

The original Titanic example uses `[X.columns]` to order the new table. It also passes the row through the original fitted scaler. The companion pipeline stores more of this preparation inside one object, but the raw input still needs the expected named columns.

14.4 The numbers differ from the slides

Check the dataset source, split proportion, random seed, stratification, selected features, and preprocessing order. Also check whether the currently fitted model was created by the intended cell. The variable `model` in a loop can refer to the last estimator visited after the loop finishes.

Library versions and numerical implementation details can also cause differences. A fixed random seed supports reproducibility under controlled conditions; it does not guarantee identical results across every version and platform. Small bar-plot error-bar differences can come from bootstrap randomness even when the underlying group rates are unchanged.

The NumPy timing demonstration is expected to vary. Timing values describe the execution environment and current run rather than a fixed property of the notebook.

14.5 Other useful checks

Symptom	What to check
<code>NameError</code>	Run the earlier cell that defines the name. A restarted runtime loses live variables.
<code>KeyError</code>	Inspect current column names and dictionary keys. Check whether an encoding or drop already removed the field.
Truth value of a Series is ambiguous	Combine Boolean Series with <code>&</code> or <code> </code> , placing each comparison in parentheses.
Array shape mismatch	Inspect both shapes. A single sample usually needs one row and the expected number of feature columns.
Missing or infinite values at fitting	Inspect the selected columns after each preparation step, including division results from ratio features.
Solver convergence warning	Check scaling, data quality, and model settings. Increasing <code>max_iter</code> can permit more optimisation steps but does not address every underlying issue.
Very large or negative housing predictions	Inspect the fitted model's form, extreme inputs, and residuals. Unconstrained linear regression can extrapolate outside meaningful target values.
Unexpected plot proportions	Check whether the plot shows counts or means, whether missing entries were excluded, and how each group was defined.
Random forest takes longer	The model trains many trees. Runtime depends on CPU resources and data size; inspect errors before assuming that a slow cell has failed.
Old output appears inconsistent with code	Rerun the edited cell and its dependent cells. Saved output can come from an earlier state.

14.6 A simple recovery procedure

Save your intended code edits, restore the required settings, restart the runtime if the state is unclear, and run from the top. Stop at the first error and address it before interpreting later outputs. Keep alternative experiments in separate variables when possible.

For a useful comparison, write down exactly what changed. “KNN with $k = 15$ on the original stratified split” is more informative than an isolated accuracy value with no description of its inputs or settings.

15 Revision questions and worked answers

Material covered: Slides 32–33; the notebook exercises and speaker-note questions.

15.1 Questions to try before reading the answers

1. What is the difference between the shape $(4, 3)$ and the shape $(4,)$?
2. Why does averaging a zero-or-one survival column produce a survival rate?
3. Why do the port counts total 889 when the table has 891 passengers?
4. What does a point beyond a box-plot whisker establish?
5. Which embarkation category is the reference in the original Titanic code?
6. What is wrong with fitting a scaler on test examples separately?
7. Can a model trained on capped house values predict above the cap?
8. Why can a training-mean baseline have a slightly negative test R^2 ?
9. Which group forms the denominator for survivor precision? Which forms it for recall?
10. What happens to already computed probabilities when you change only a decision threshold?
11. Did the three housing ratios improve the linear regression result in this exercise?
12. Why does selecting the best k from test results affect the status of that test set?

15.2 Worked answers

- 1. Dimensions.** A $(4, 3)$ array has four rows and three columns. A $(4,)$ array has one dimension containing four values. Shape describes organisation as well as the total number of entries.
- 2. A binary mean.** Summing zero-or-one outcomes counts the ones. Dividing by the number of rows gives the fraction with outcome one.
- 3. Missing codes.** Two embarkation codes are missing, and `value_counts()` excludes missing entries by default. Always check what population a summary includes.
- 4. A plotting-rule outlier.** It is unusually far from the central quartiles under the whisker rule. Further inspection is required to decide whether it is an error, a valid rare value, or evidence of another subgroup.
- 5. Port C.** The retained dummy columns are Q and S. A row with both equal to zero represents C under this known-category encoding.
- 6. Inconsistent coordinates and test information.** A separate test-fitted scaler defines different means and scales and learns from the evaluation set. Use the training-fitted transformation consistently.
- 7. Yes, for a suitable model.** Ordinary linear regression can produce values beyond the observed cap. The missing information concerns the true uncapped targets, so the data cannot directly teach those distinctions.

8. Different means. The training mean used by the baseline can differ from the evaluation-set mean used in the denominator of R^2 . A small negative value can result and round to -0.00 .

9. Different reference groups. Precision divides true-positive survivors by all passengers predicted to survive. Recall divides true-positive survivors by all actual survivors.

10. The probabilities remain fixed. The threshold converts the same scores into different class decisions. Refitting is a separate operation.

11. No improvement in this comparison. Removing the ratios gives a lower RMSE, approximately \$70,061 compared with \$72,669, and a higher R^2 , approximately 0.63 compared with 0.60. The effect should be interpreted for this model and split.

12. The results guide a choice. Once test results determine the neighbour count, those examples influence development. Use validation to make the choice and evaluate the selected workflow on separate held-out data.

15.3 Questions beginners commonly ask

Why use a median for missing age? It provides a simple, reproducible central value without allowing a few extreme magnitudes to pull it strongly. It preserves rows with other useful information. It also changes the age distribution and leaves uncertainty about the missing ages, so it is a starting method to evaluate.

Does every numerical column belong in the model? A number can be an identifier, an encoded category, a measurement, or even the answer itself. Choose inputs according to their meaning and availability at prediction time. Numerical storage alone does not make a field a valid feature.

Does a stronger correlation guarantee a better model? Correlation describes one kind of pairwise association. A model combines information under its own assumptions, and its measured performance depends on the evaluation data. The housing ratio exercise provides a concrete example of this distinction.

Is an 80-percent classification accuracy good? Compare it with the baseline, class counts, important errors, and intended use. In this notebook it improves substantially on a 61.5-percent constant baseline, but it also misses 22 of 69 survivors. Both facts matter.

Can a test score exceed the training score? Yes. The sets contain different examples, and finite samples can differ in difficulty. The KNN result at $k = 15$ shows a small reverse gap in this exercise.

Why are some algorithms evaluated with scaling and others without? Scaling changes distances and can affect regularised optimisation. Conventional trees depend on within-feature ordering and usually do not need it. Keep each model's preprocessing within a reproducible workflow and compare on the same intended evaluation examples.

Can I start with my own dataset? Yes. Define its rows, columns, units, target, and data source; load it; and apply the same reasoning. A new dataset may require a different split, missing-value treatment, encoding, or metric. Copy the purpose of each step and adapt the details to the new information.

15.4 Before Day 3

1. Complete all three Day 2 notebooks from a clean runtime.
2. Reproduce the voltage and exam-score exercises and explain the two axis choices aloud.

3. Choose a Titanic plot and describe its axes, denominator, missing-value handling, and main pattern.
4. Recalculate accuracy, precision, recall, and F1 from the four confusion-matrix counts without using a metric function.
5. Try one additional feature and record the exact preparation, split, and metrics used. Treat this as development work.
6. Compare the original workflow with the training-only pipeline and explain why the preparation boundary matters.
7. Bring one question about model complexity or a comparison that gave an unexpected result.

Day 3 extends these ideas through further models, validation, overfitting controls, clustering, and dimensionality reduction. The data structures and evaluation habits from today support that next stage.

16 Glossary, coverage guide, and sources

16.1 Day 2 glossary

Term	Meaning
Array	An organised numerical collection with a shape and a data type.
Axis	A dimension along which an array operation acts or reduces values.
Baseline	A simple reference predictor evaluated under the same intended conditions.
Boolean mask	A collection of true-or-false values used to select entries or rows.
Broadcasting	Rules for applying compatible NumPy operations across differently shaped inputs.
Calibration	Agreement between predicted probabilities and observed frequencies in suitable evaluated groups.
Categorical variable	A variable whose values identify categories. Its codes do not automatically express meaningful numerical distances.
Coefficient	A fitted weight in a model's prediction expression.
ColumnTransformer	A tool that applies named preparation steps to selected groups of columns and combines their outputs.
Confusion matrix	Counts of true-label and predicted-label combinations.
Correlation	A measure of association; Pearson correlation specifically measures linear association.
DataFrame	A labelled Pandas table.
Data leakage	Use during model development of information that should be unavailable under the intended evaluation or prediction setting.
EDA	Exploratory data analysis using inspection, summaries, plots, and context.
Encoding	Converting a representation, such as a category, into a form the selected model can use.
F1	The harmonic mean of positive-class precision and recall.
False negative	A truly positive example assigned a negative prediction.
False positive	A truly negative example assigned a positive prediction.

Term	Meaning
Feature engineering	Constructing input descriptions using existing data and domain understanding.
Imputation	Filling missing entries using a specified rule or fitted method.
Intercept	The constant term in a model's linear score or prediction.
IQR	Interquartile range, equal to the third quartile minus the first.
MAE	Mean absolute error, measured in the target's units.
Median	The central value of an ordered distribution under the chosen quantile convention.
Mode	A most frequently occurring value.
One-hot encoding	Representing categories with indicator columns.
Overfitting	Adapting too closely to training-specific structure so that performance transfers poorly.
Pipeline	A sequence that fits and reuses preprocessing steps together with an estimator.
Precision	Fraction of positive predictions that are truly positive.
Quartile	A boundary describing quarters of an ordered distribution.
R^2	Squared-error reduction relative to the evaluation-target-mean reference, under its usual non-constant-target definition.
Recall	Fraction of actual positive examples that are identified as positive.
Regularisation	A restriction or penalty that influences model complexity or parameter values during fitting.
Residual	A difference between a prediction and its target, with a stated sign convention.
RMSE	Square root of mean squared error, measured in the target's units.
Series	A one-dimensional labelled Pandas collection.
Sigmoid	The function $1/(1 + e^{-z})$ mapping a score to a value between zero and one.
Standardisation	Subtracting a fitted mean and dividing by a fitted scale, usually the training standard deviation.
Stratification	Splitting with an attempt to preserve specified group or class proportions.
Support	Number of true examples of a class in an evaluation set.
Threshold	A chosen score or probability boundary used to make a decision.
Top-coding	Recording all values above a limit at a common maximum, losing distinctions above that limit.
Vectorisation	Expressing operations over an array so the library performs the repeated numerical work.

16.2 Coverage of the supplied files

Material	Explanation in this document
Slides 1–3	Day 2 goals, recap, notebook map, and session sequence.
Slides 4–8	NumPy and Pandas operations, all Notebook 1 examples, and both exercises.
Slides 9–13	EDA questions, Titanic fields, six plots, and observed group rates.
Slides 14–19	Missing values, target leakage, engineered features, category encoding, scaling, and splitting.
Slides 20–23	Housing context, EDA, preparation, regression, metrics, model comparisons, and all housing exercises.
Slides 24–30	Logistic regression, KNN, trees, confusion counts, classification metrics, overfitting, new passengers, and exercises.
Slides 31–33	The scikit-learn recipe, revision, questions, and preparation for Day 3.
Notebook 1	All non-empty code examples and both solution cells are explained in the NumPy and Pandas chapters.
Notebook 2	All EDA, preparation, fitting, plotting, new-prediction, and solution cells are covered.
Notebook 3	All EDA, preparation, helper, model, diagnostic-plot, and solution cells are covered.
Speaker notes	Teaching examples, intended interpretations, reported results, questions, troubleshooting, and terminology are integrated throughout.

16.3 Important clarifications carried into the explanations

The original Titanic encoder drops C and retains Q and S. The under-18 count and the under-16 survival percentage describe different groups. Learned imputation and encoding should be fitted within training data, alongside the correctly training-fitted scalars. A random timing demonstration has no fixed speed multiplier.

The housing target represents district medians and contains a cap. Linear predictions can exceed that cap. The constant training-mean baseline can have slightly negative test R^2 . The ratio-feature experiment worsens the checked linear result, and extreme ratios are a plausible issue to investigate rather than an isolated causal finding.

Model-comparison tables refer to particular rows and settings. A training–test gap is a warning sign to investigate, threshold changes do not refit the model, and precision is not guaranteed to rise at every higher threshold. These details help turn short slide statements into accurate working explanations.

16.4 Source files and computational verification

The main sources are the five supplied Day 2 files:

1. *Day 2: Data, EDA & Supervised Learning*, 33-slide PDF.
2. `Day2_1_NumPy_Pandas.ipynb`.
3. `Day2_2_Titanic_EDA_to_Classification.ipynb`.
4. `Day2_3_Housing_Regression.ipynb`.
5. `day2_speaker_notes.pdf`.

The non-empty notebook code cells were executed in their original order, using downloaded copies of the CSV sources named by their loaders. The same source data was used for the figures and checked numerical outputs. No uploaded source notebook was edited.

The verification environment used NumPy 2.3.5, Pandas 2.2.3, Seaborn 0.13.2, and scikit-learn 1.8.0. The main results reproduced the provided materials. Small visual, timing, or numerical differences can occur in other environments or after code changes.

16.5 Dataset locations and supplementary documentation

The data sources referenced by the supplied notebooks are:

- Seaborn Titanic CSV: <https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv>.
- Housing CSV used in Notebook 3: <https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv>.

Official supplementary documentation was consulted for specific interface and evaluation details:

- Pandas, *get_dummies*: https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html. The meaning of dropping the first category level.
- Seaborn, *barplot*: <https://seaborn.pydata.org/generated/seaborn.barplot>. The default estimator and confidence-interval display.
- scikit-learn, *Common pitfalls and recommended practices*: https://scikit-learn.org/1.8/common_pitfalls.html. Training-only fitted preparation and pipelines.
- scikit-learn, *R-squared score*: https://scikit-learn.org/1.6/modules/generated/sklearn.metrics.r2_score.html. The interpretation of zero and negative scores.